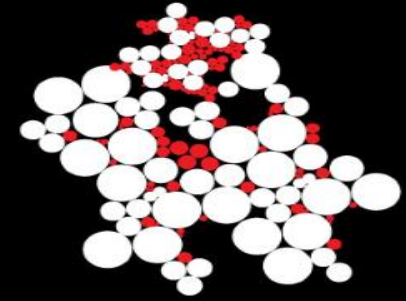


UNIVERSITY OF TWENTE.



Automated Performance Prediction of Medical Systems

Freek van den Berg, Anne Remke and Boudewijn R. Haverkort

Computing Response Time Distributions via Model Checking

Freek van den Berg, Jozef Hooman, Arnd Hartmanns, Boudewijn R. Haverkort and Anne Remke



EPEW 2015, Madrid, Spain
September 1st, 2015



COMMIT/ALLEGIO

Embedded Systems
Innovation
BY TNO

PHILIPS
sense and simplicity

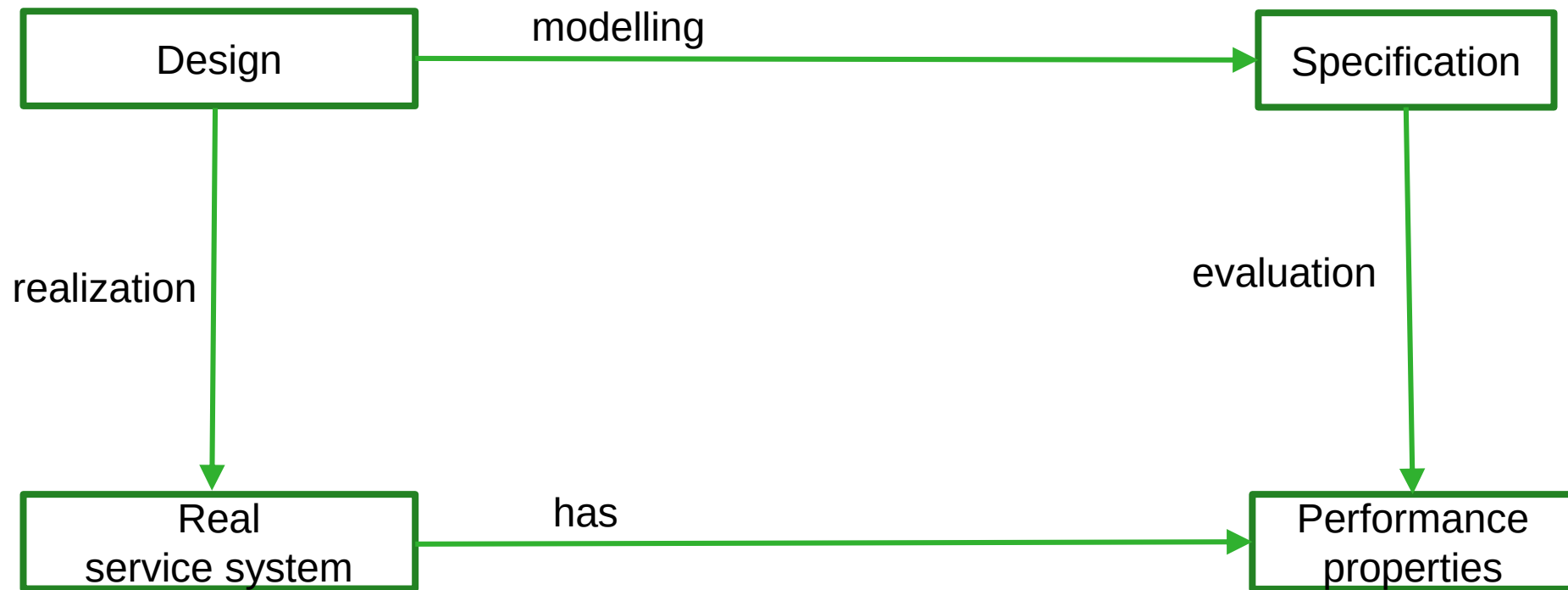
Outline

- Previous work: The iDSL language and tool chain
- Paper 1: Automated Performance Prediction of Medical Systems
- LUNCH
- Paper 2: Computing Response Time Distributions via Model Checking
- Future work: ``Computing Response Time Distributions 2.0''

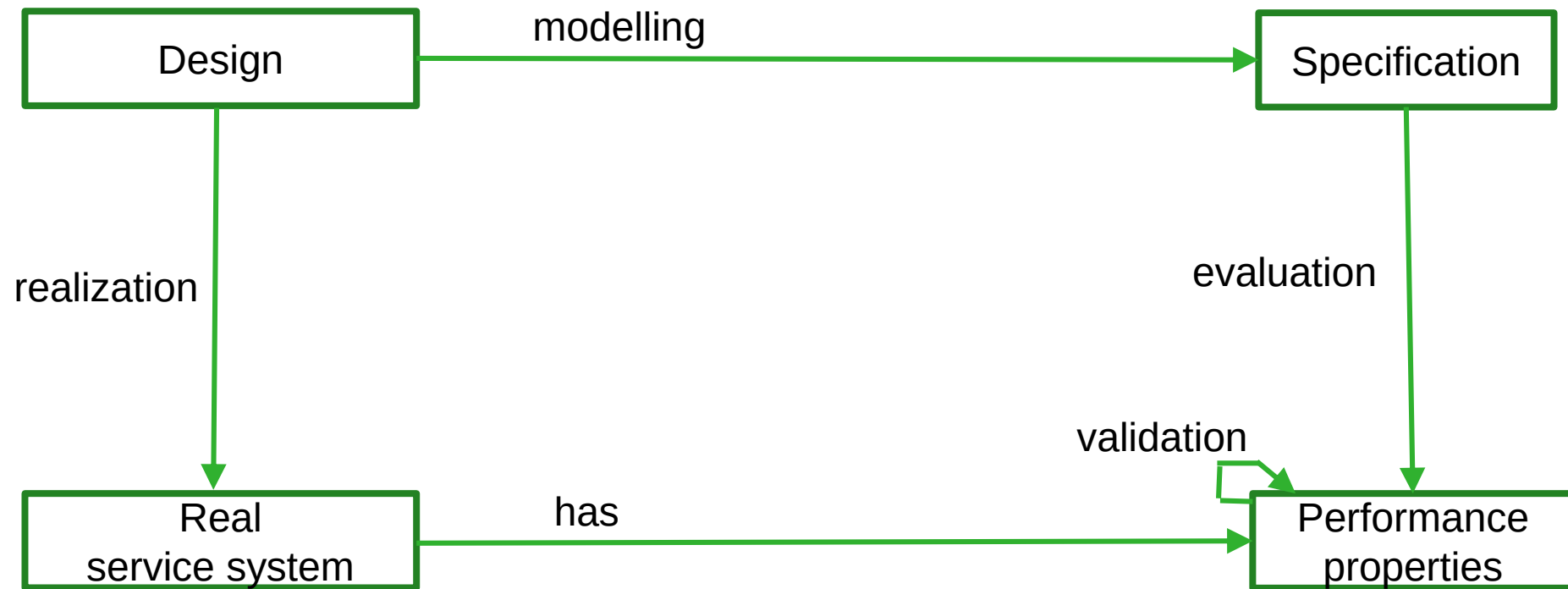
Performance evaluation of service systems



Performance evaluation of service systems



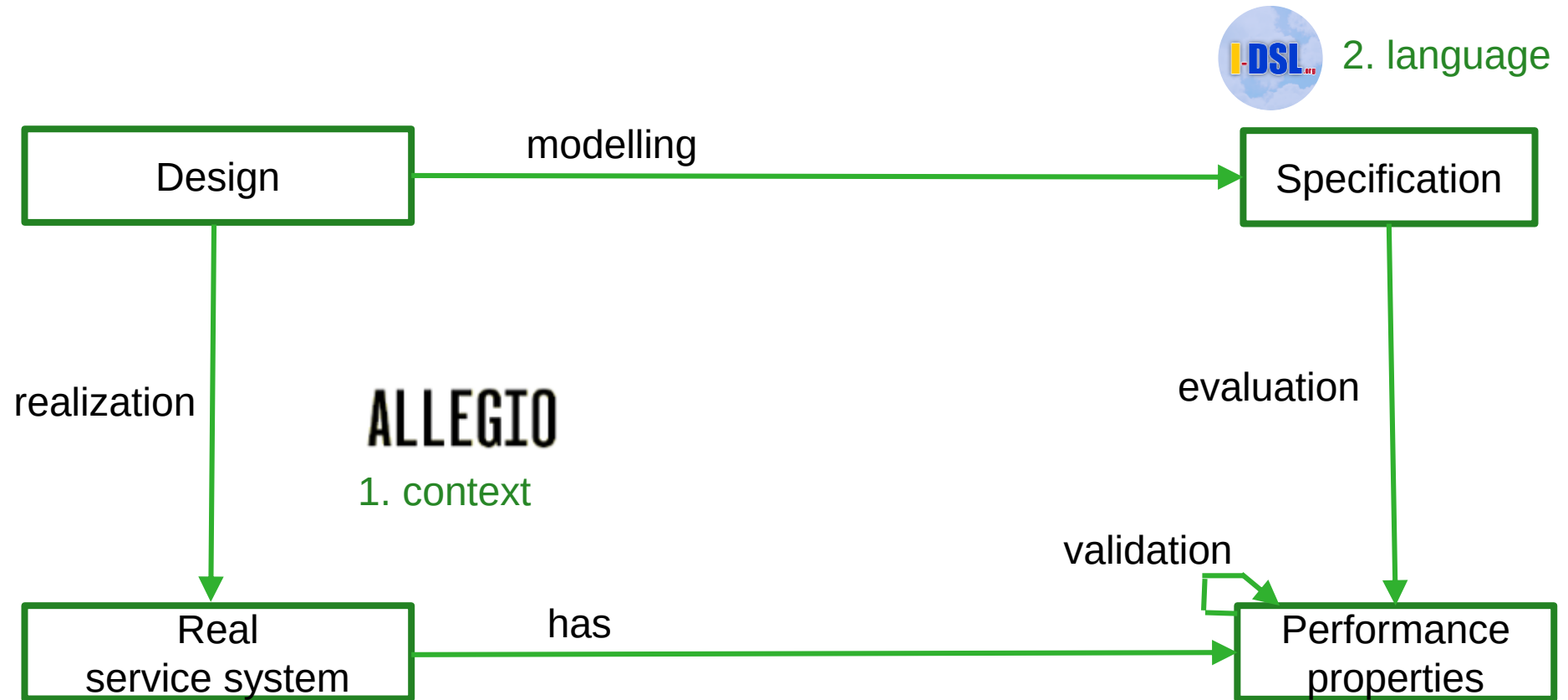
Performance evaluation of service systems



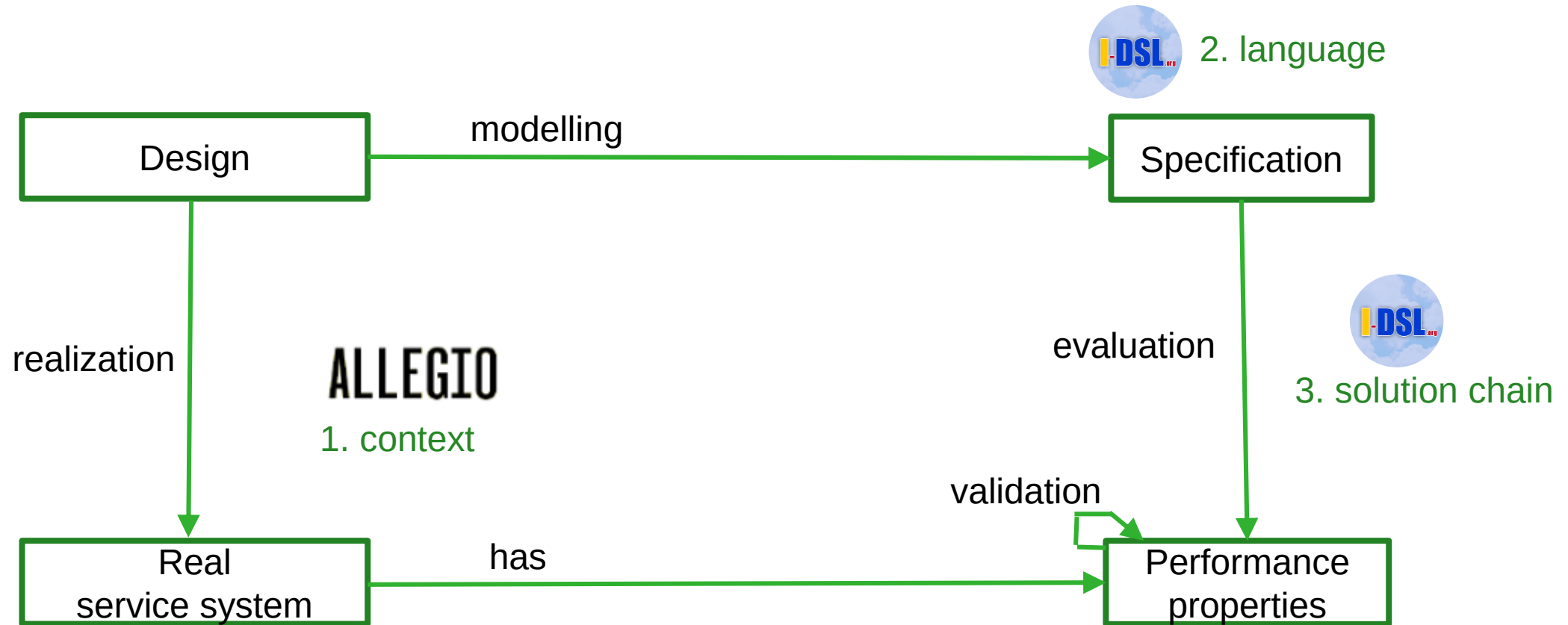
Performance evaluation of service systems



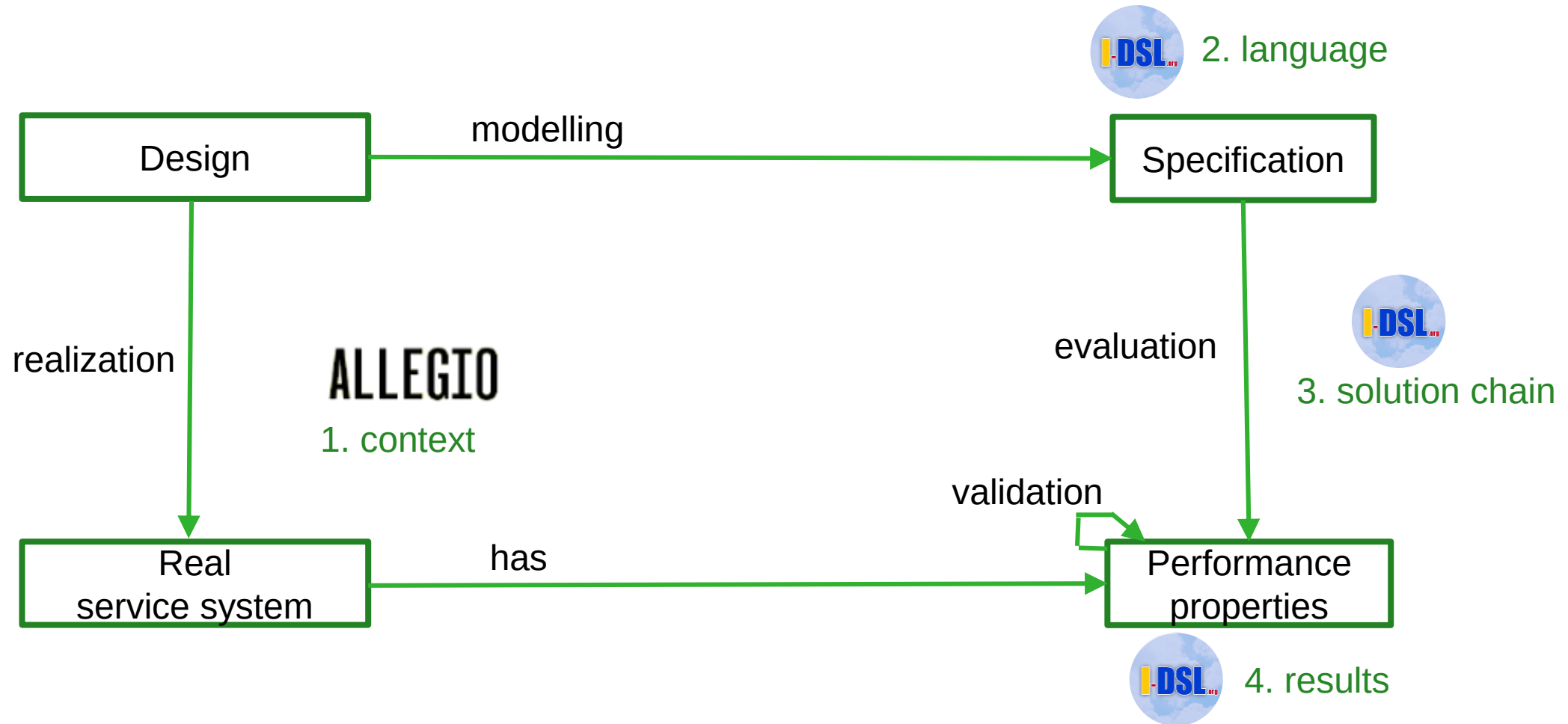
Performance evaluation of service systems



Performance evaluation of service systems



Performance evaluation of service systems



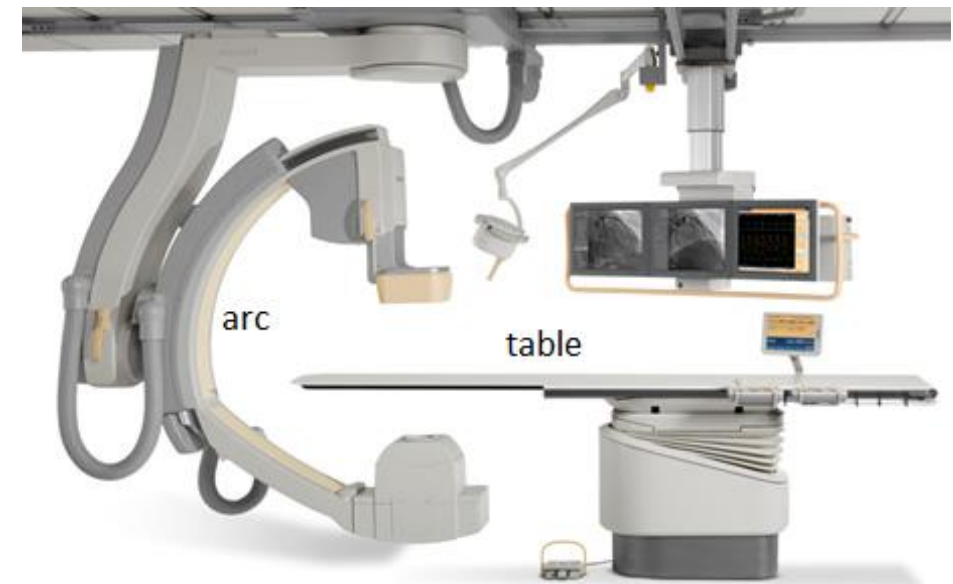
Outline

- Previous work: The iDSL language and tool chain*
- Paper 1: Automated Performance Prediction of Medical Systems
- LUNCH
- Paper 2: Computing Response Time Distributions via Model Checking
- Future work: ``Computing Response Time Distributions 2.0''

* Freek van den Berg, Anne Remke, Boudewijn R. Haverkort:
A Domain Specific Language for Performance Evaluation of Medical Imaging Systems.

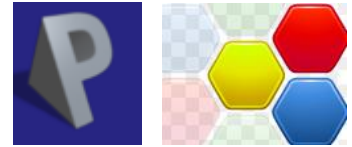
Context

- Medical Imaging Machines (MIS) perform safety critical tasks
- iXR machines are MIS that
 - Dynamically record X-ray based images
 - Used by surgeons while operating a patient
 - Are designed on the basis of measurements
- Surgeons demand up-to-date images
 - Performance is key for iXR systems



Approach

- Toolsets for modelling and analyzing stochastic systems
 - PRISM + Modest: multiple formalisms and ways of analysis, advanced algorithms



Approach

- Toolsets for modelling and analyzing stochastic systems
 - PRISM + Modest: multiple formalisms and ways of analysis, advanced algorithms
- Drawbacks
 - Hard to use for system designers
 - Low level code: large models, redundancy
 - One model does **not** fit all
 - Limited support for multiple designs, and visualizations



Approach

- Toolsets for modelling and analyzing stochastic systems
 - PRISM + Modest: multiple formalisms and ways of analysis, advanced algorithms

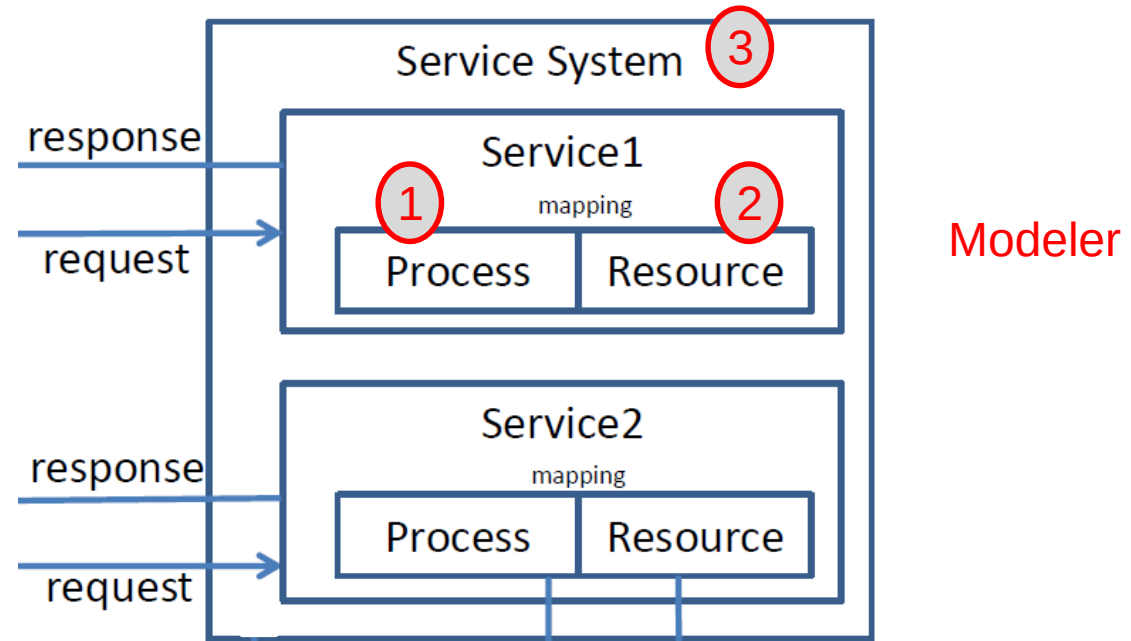


- Drawbacks
 - Hard to use for system designers
 - Low level code: large models, redundancy
 - One model does **not** fit all
 - Limited support for multiple designs, and visualizations

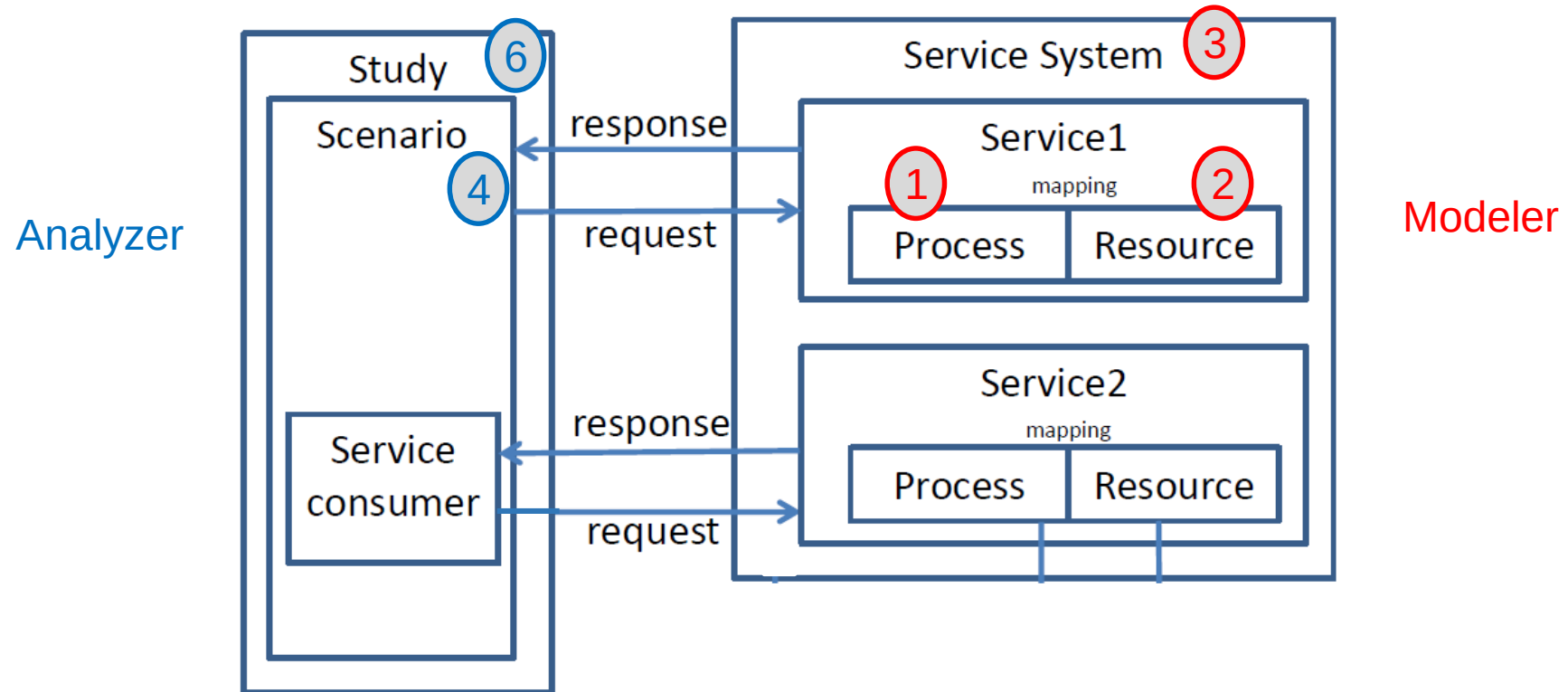
- Our approach
 - A high level language and solution chain for service systems
 - Delegate low-level analysis to Modest



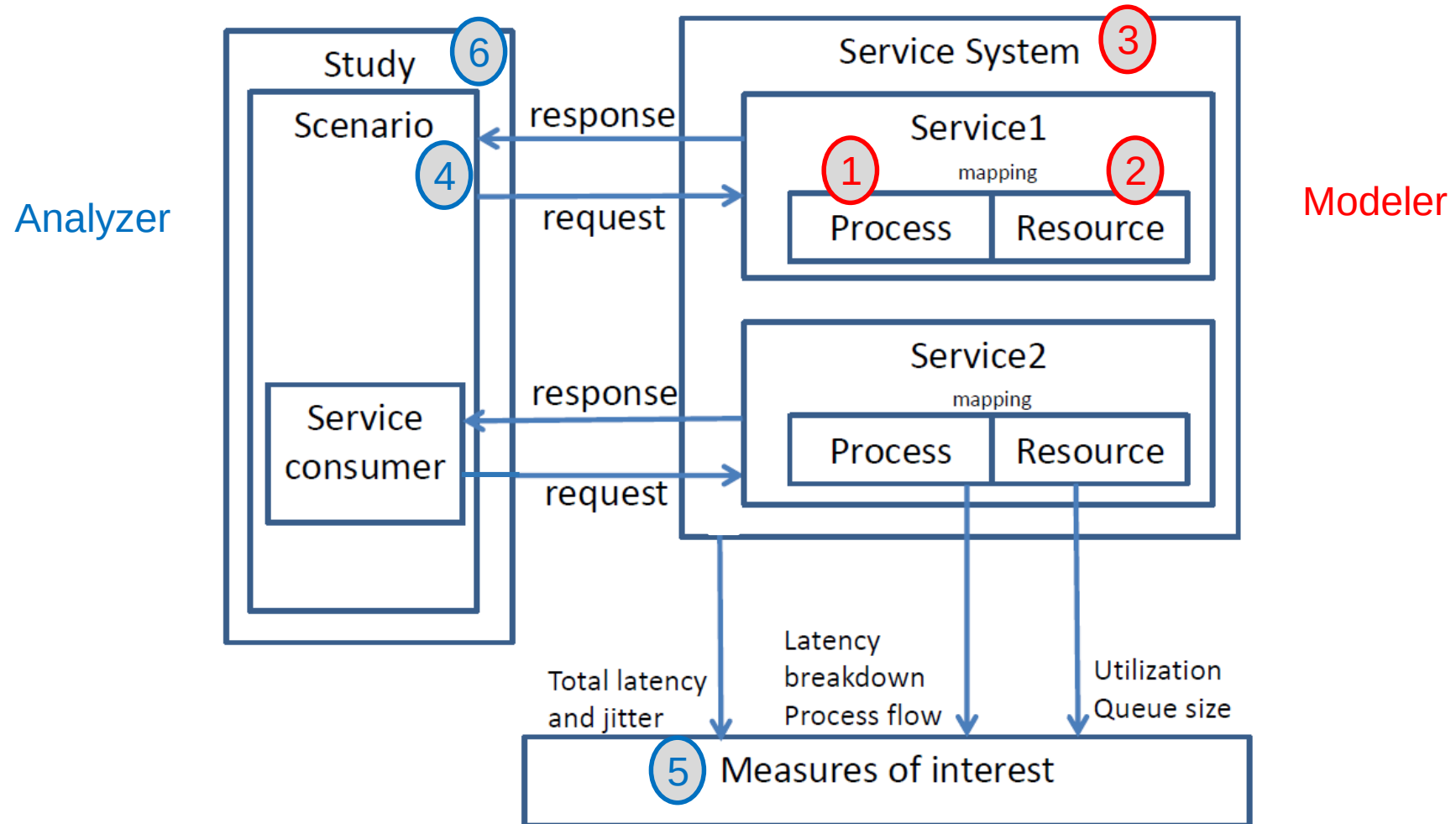
iDSL language



iDSL language



iDSL language



iDSL language instance: pseudo code

Section System

Service image_processing_service ...

Process

image_processing_seq ...

Resource

image_processing_PC

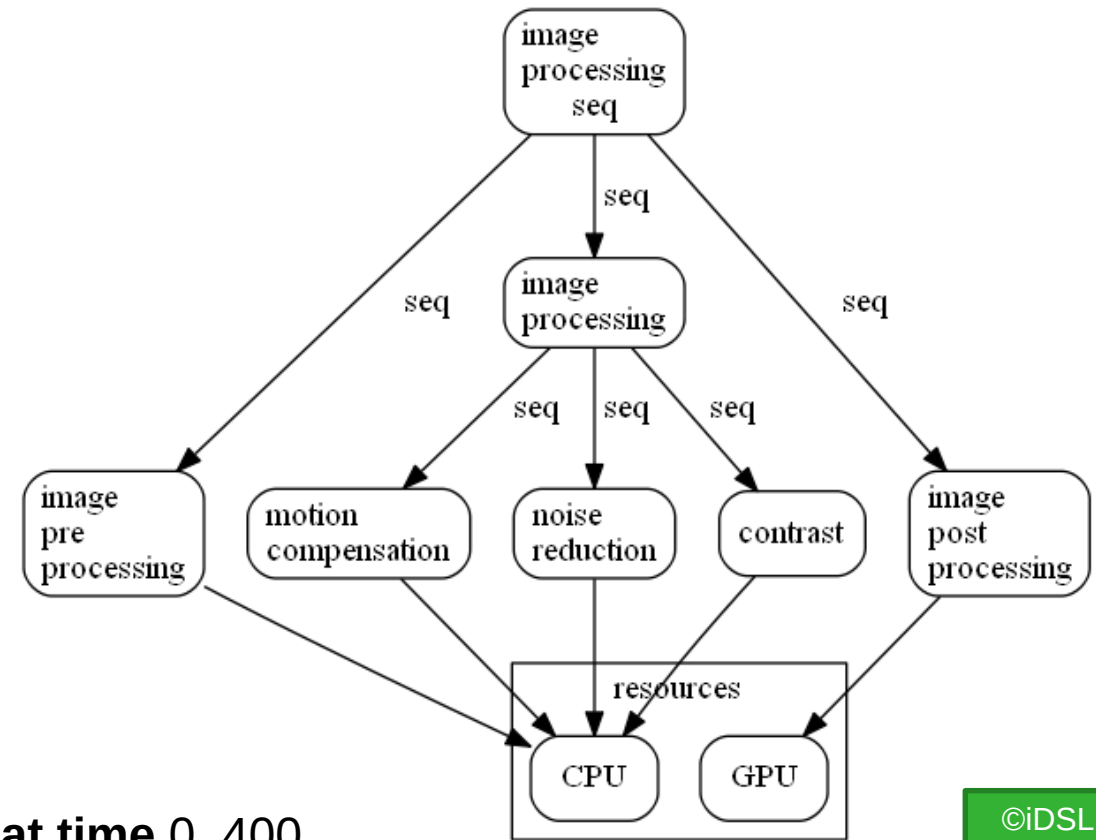
Mapping assign {

(image_pre_processing, CPU)
(motion_compensation, CPU)
(noise_reduction, CPU)
(contrast, CPU)
(image_post_processing, GPU)

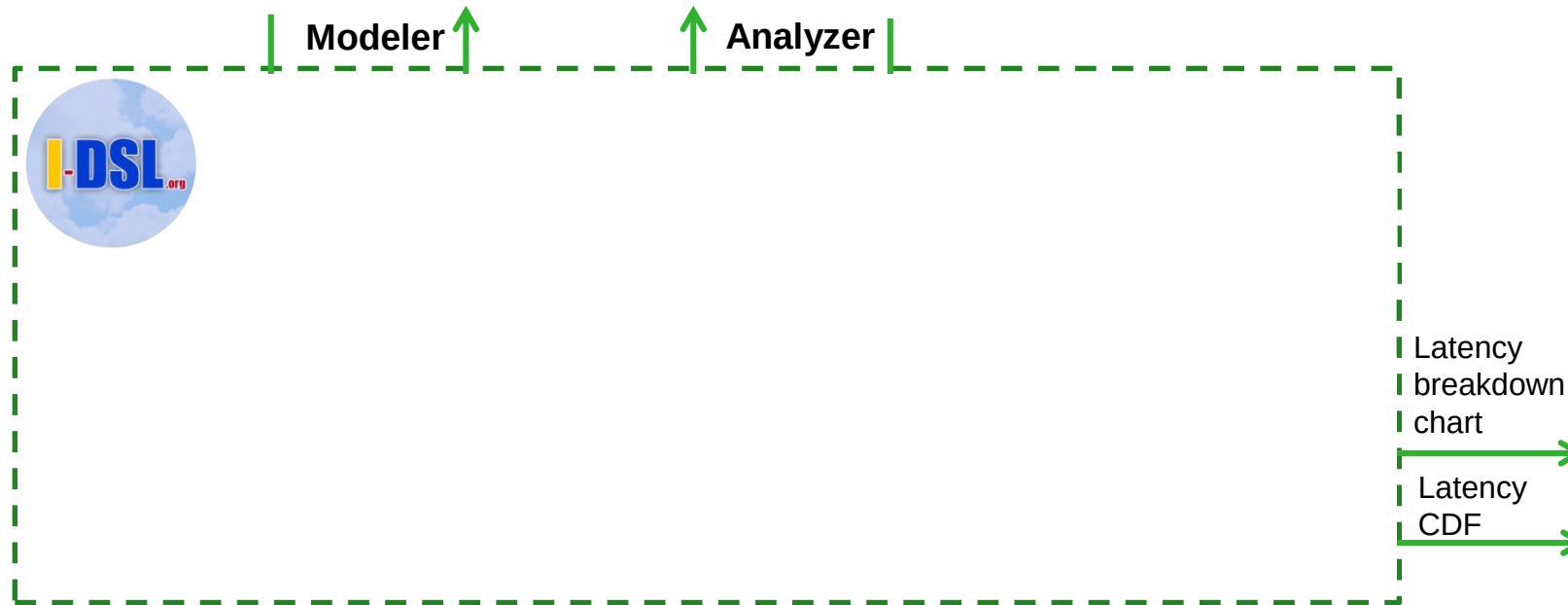
}

Scenario image_processing_run

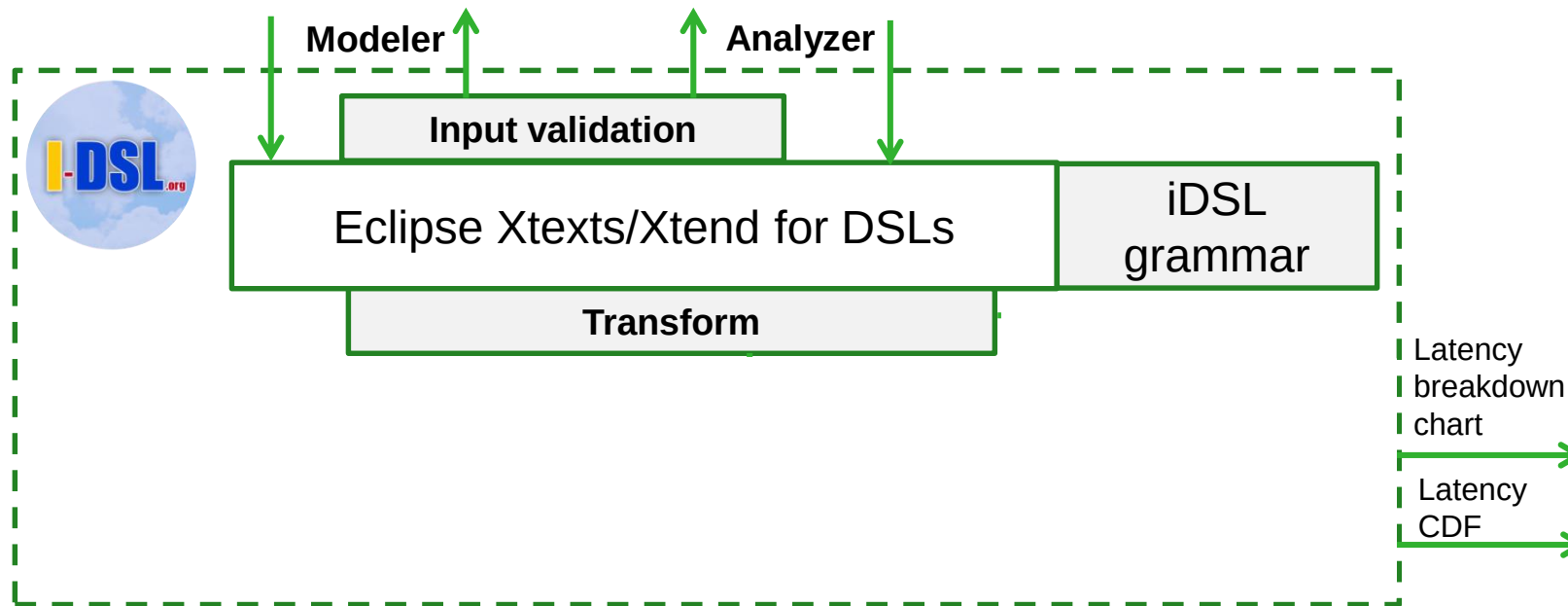
ServiceRequest image_processing_service at time 0, 400, ...



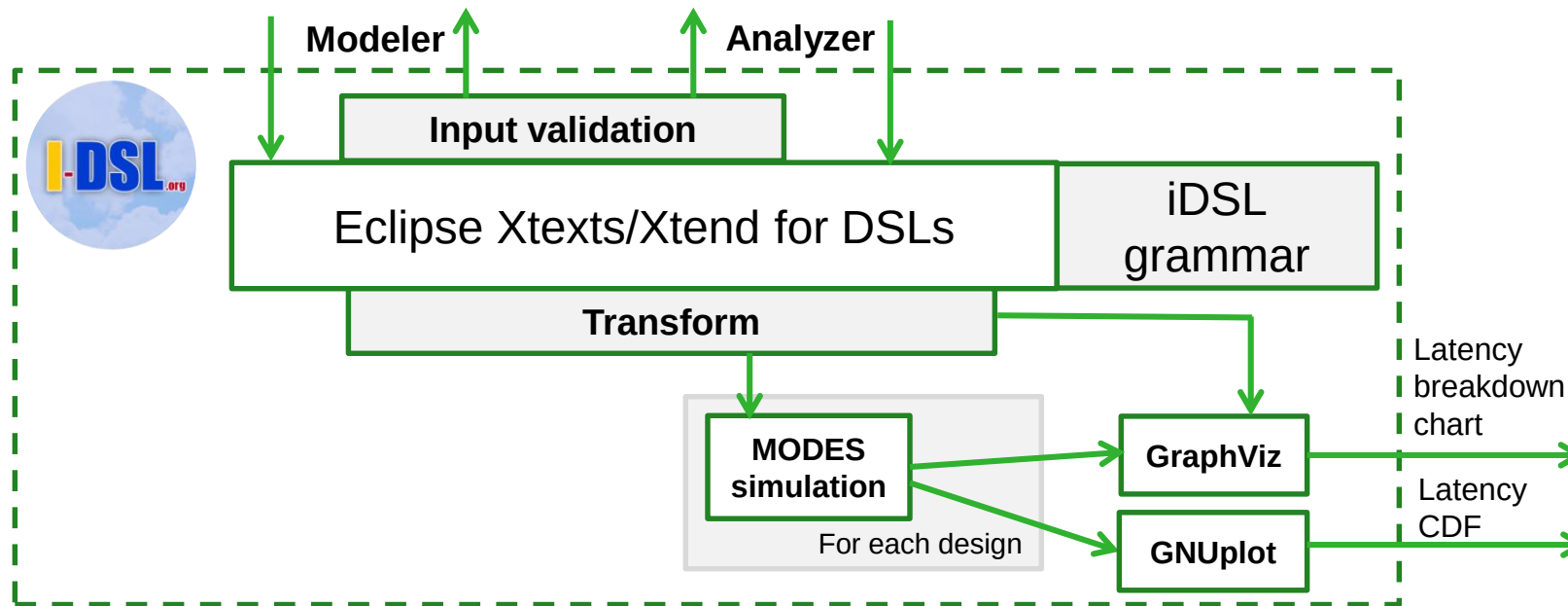
iDSL solution chain



iDSL solution chain

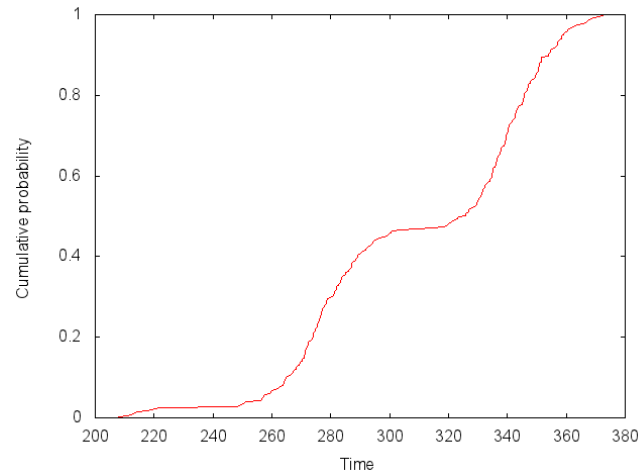


iDSL solution chain



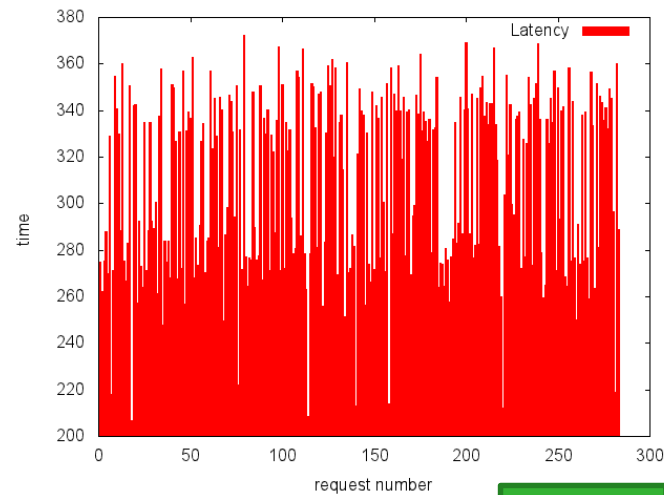
iDSL solution chain output

CDF



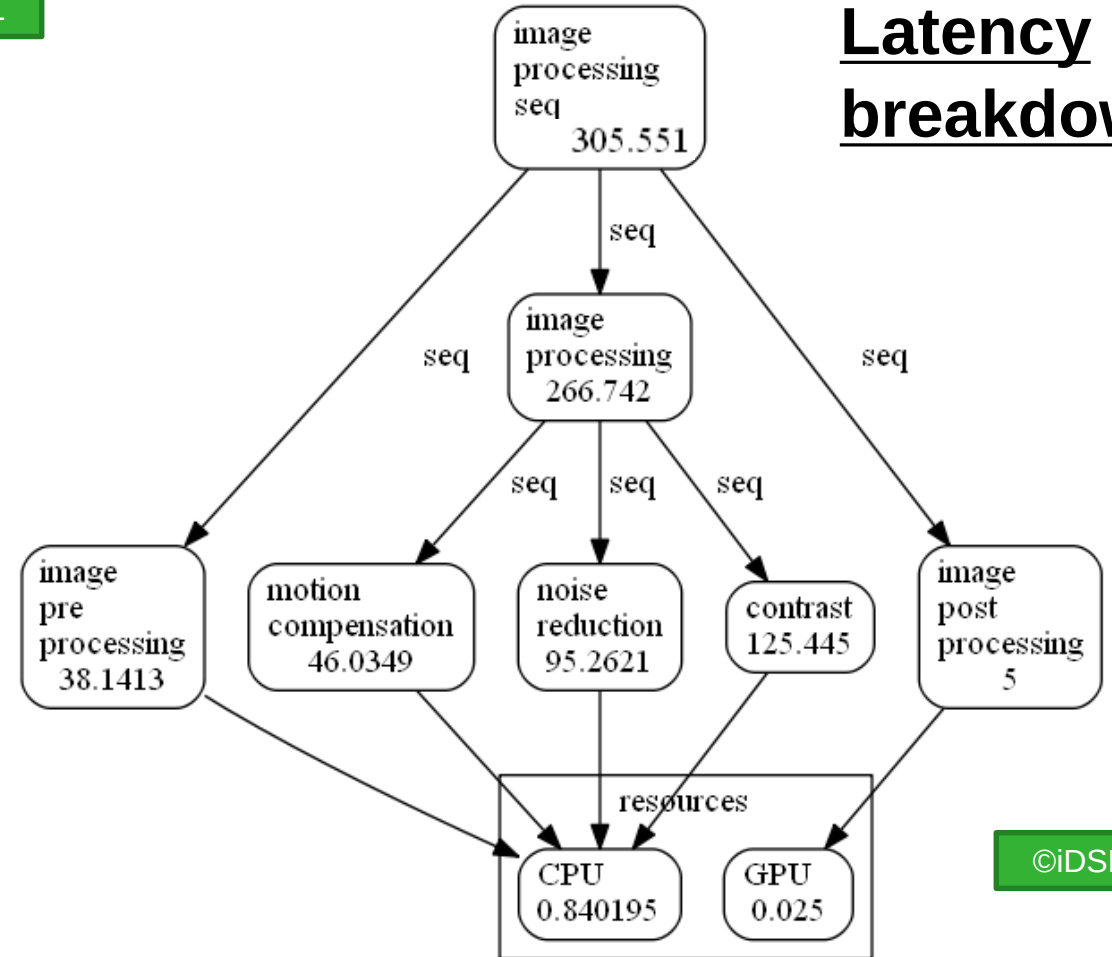
©iDSL

Latency chart



©iDSL

Latency breakdown



©iDSL

Outline

- Previous work: The iDSL language and tool chain
- Paper 1: Automated Performance Predication of Medical Systems*
- LUNCH
- Paper 2: Computing Response Time Distributions via Model Checking
- Future work: ``Computing Response Time Distributions 2.0''

* Freek van den Berg, Anne Remke, Boudewijn R. Haverkort:
iDSL: Automated Performance Prediction and Analysis of Medical Imaging Systems.

Context

- Evaluate the performance of iXR machines
 - Do biplane (3D) iXR systems perform as good on shared hardware as on dedicated hardware?
 - Design space: resolution, mono- or biplane buffer size, fps
 - Real measurements available

Context

- Evaluate the performance of iXR machines
 - Do biplane (3D) iXR systems perform as good on shared hardware as on dedicated hardware?
 - Design space: resolution, mono- or biplane buffer size, fps
 - Real measurements available



Context

- Evaluate the performance of iXR machines
 - Do biplane (3D) iXR systems perform as good on shared hardware as on dedicated hardware?
 - Design space: resolution, mono- or biplane buffer size, fps
 - Real measurements available
- Insights in performance needed
 - Image latencies and trade-offs, e.g., determining buffer sizes



Context

- Evaluate the performance of iXR machines
 - Do biplane (3D) iXR systems perform as good on shared hardware as on dedicated hardware?
 - Design space: resolution, mono- or biplane buffer size, fps
 - Real measurements available
- Insights in performance needed
 - Image latencies and trade-offs, e.g., determining buffer sizes
- Approach
 - Simulations for **many designs** using **few measurements**
 - Leading to **understandable results**, in a **fully automatic** manner



iDSL model: process, resource, system

Section Process

```
ProcessModel Image_Processing mutex { seq {  
    seq Noise_reduction {  
        atom Pre_processing load ... // depends on resolution and mode  
        atom Basic load ...  
        ...  
    }  
}}
```

Section Resource

```
ResourceModel Biplane_PC decomp { atom CPU rate 1 }
```

Section System

```
Service Image_Processing_Service  
Process Image_Processing  
Resource Biplane_PC  
Mapping assign { (* ,CPU) } scheduling policy { (CPU, FIFO) }
```

iDSL model: scenario, measure, study

Section Scenario

Scenario BiPlane_Image_Processing_run

ServiceRequest Image_Processing_Service at time 0, (1/dspace("framerate")), ...

Section Measure

Measure ServiceResponseTimes using 1 runs of 50 ServiceRequests

Section Study

Scenario BiPlane_Image_Processing_run

DesignSpace

("resolution" {"512" "1024" "2048" })
("mode" {"monoplane" "biplane"})
("framerate" {"5" "10" "15" "30" "60" "120" "240"})
("numinstances" {"1" "2" "3" "4"})

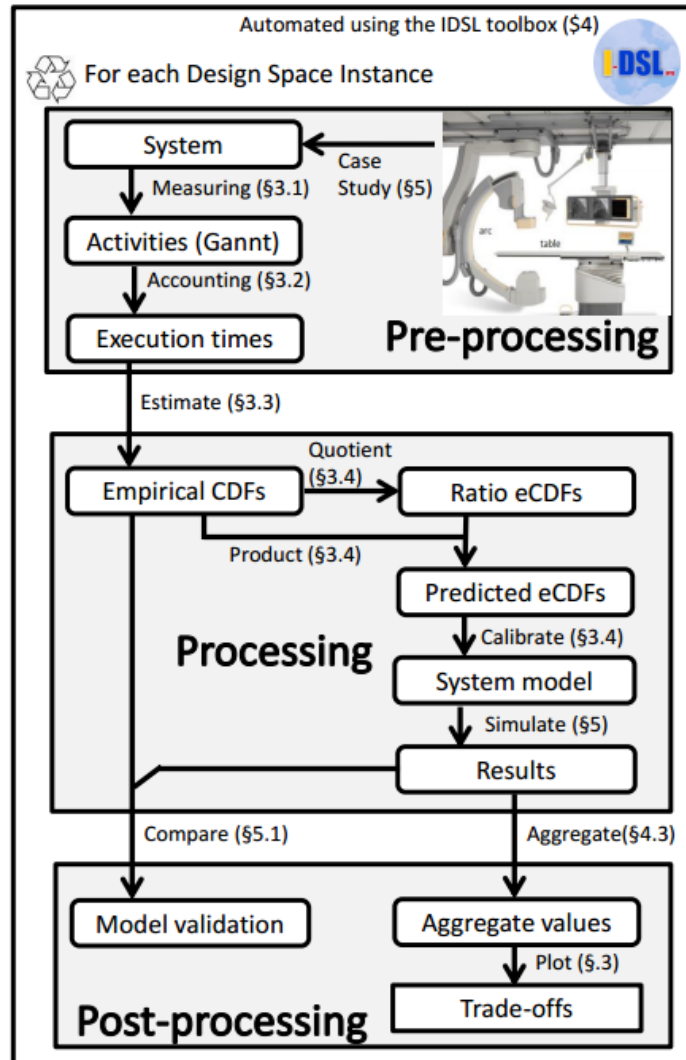
•

○

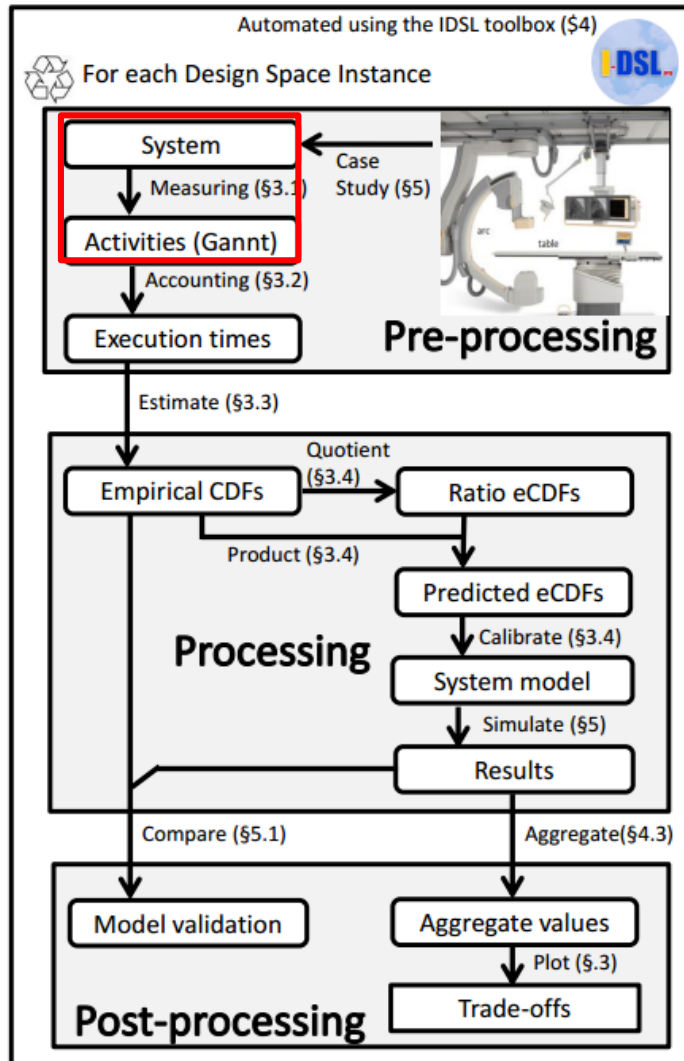
○

$3 * 2 * 7 * 4 =$
168 designs

iDSL solution chain of the approach

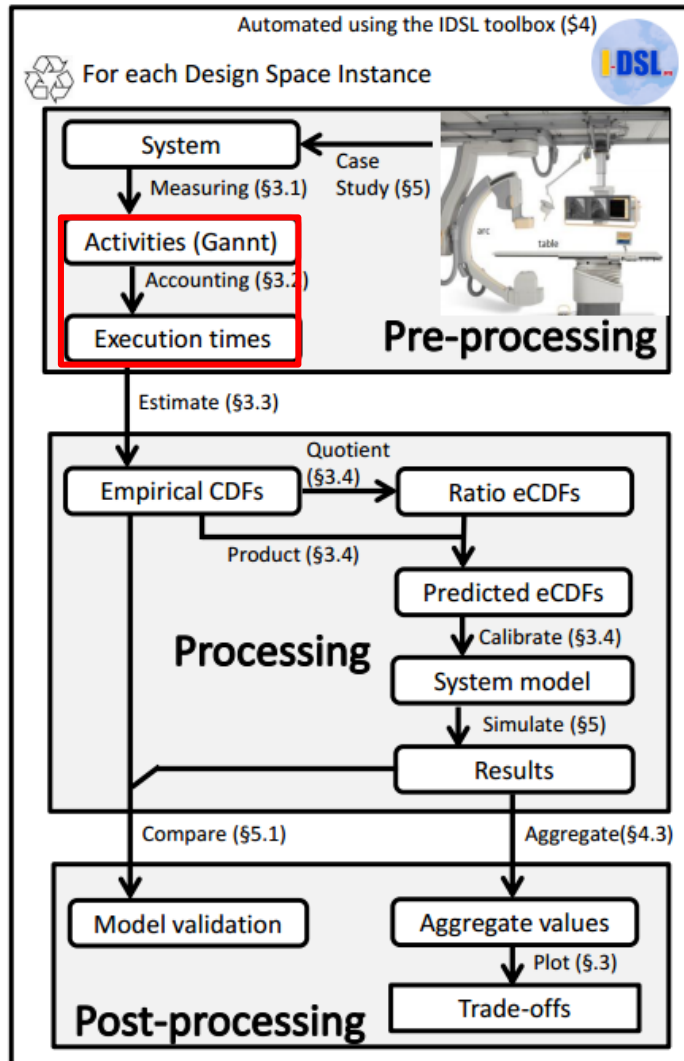


iDSL solution chain of the approach

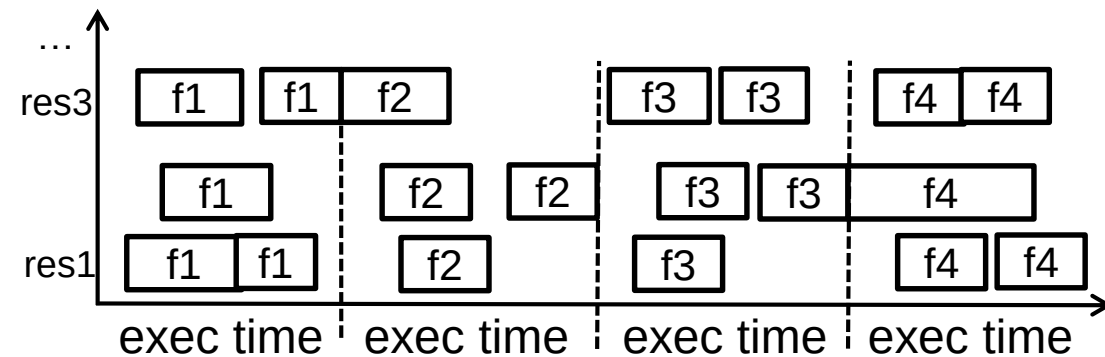


Measurements are taken by adding stopwatches to the program code

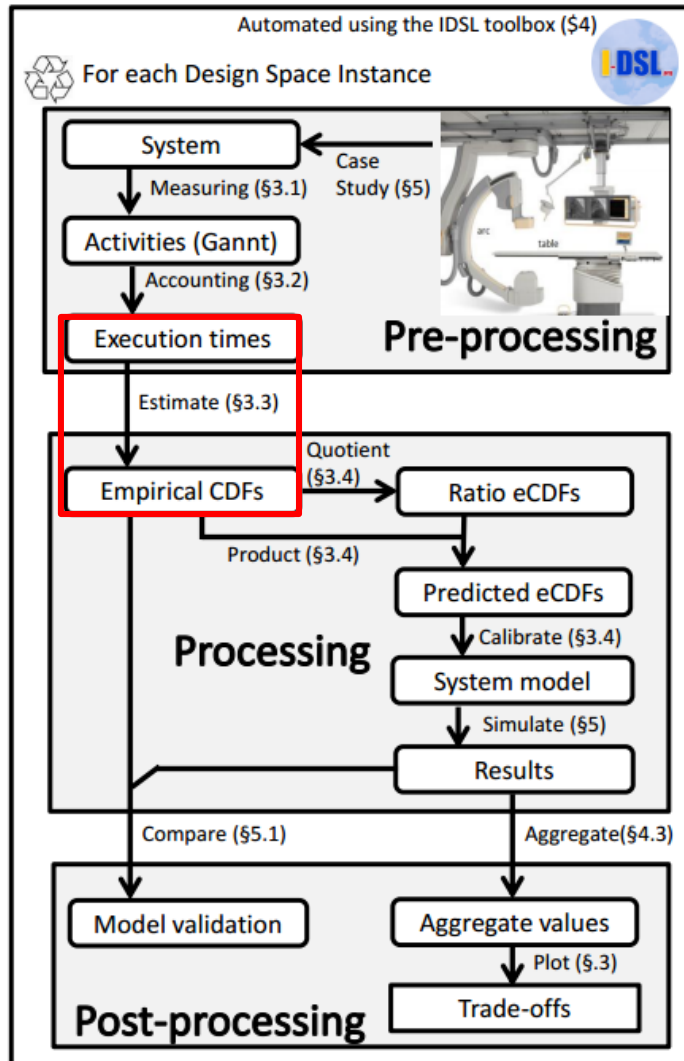
iDSL solution chain of the approach



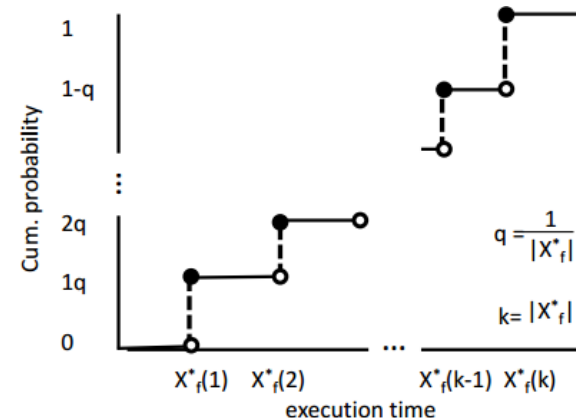
These measurements are converted into execution times



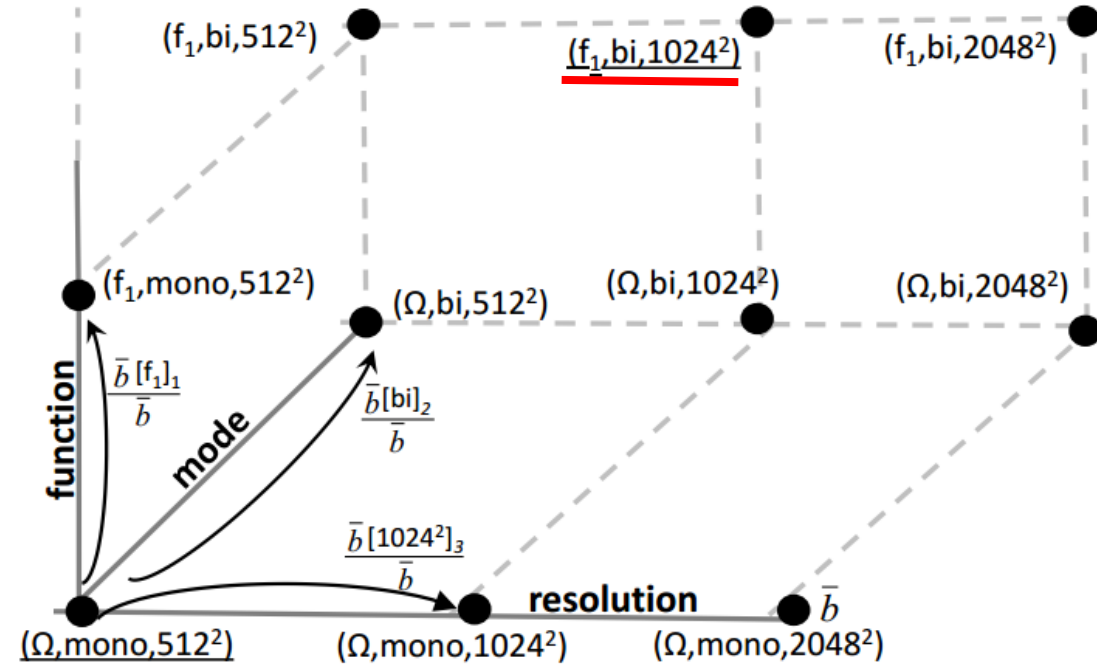
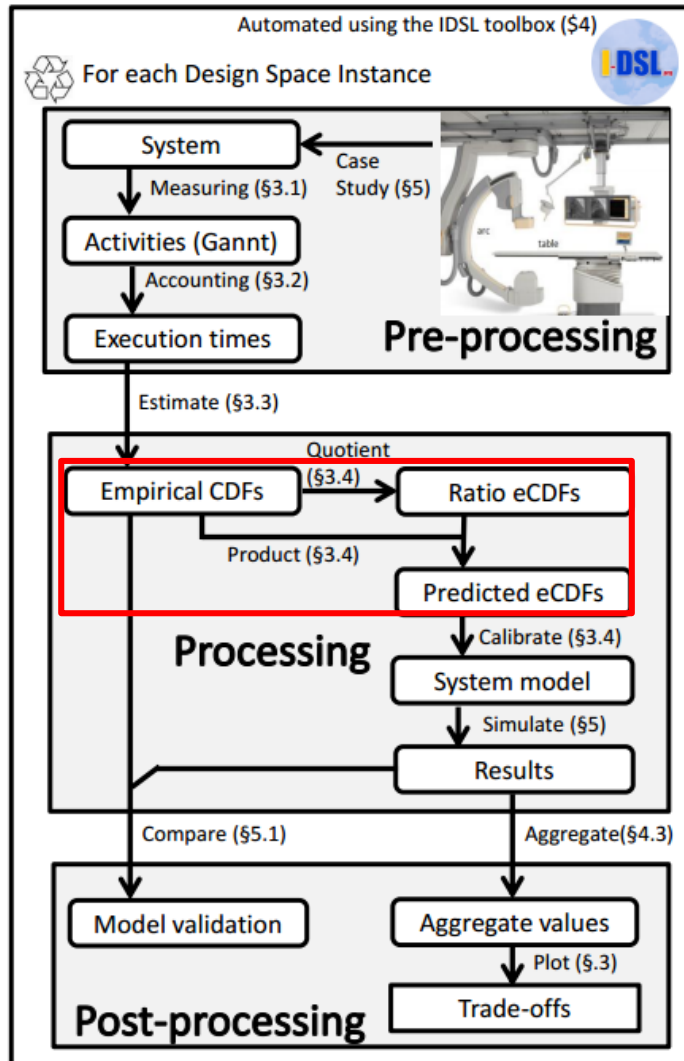
iDSL solution chain of the approach



Execution times form the basis of empirical CDFs



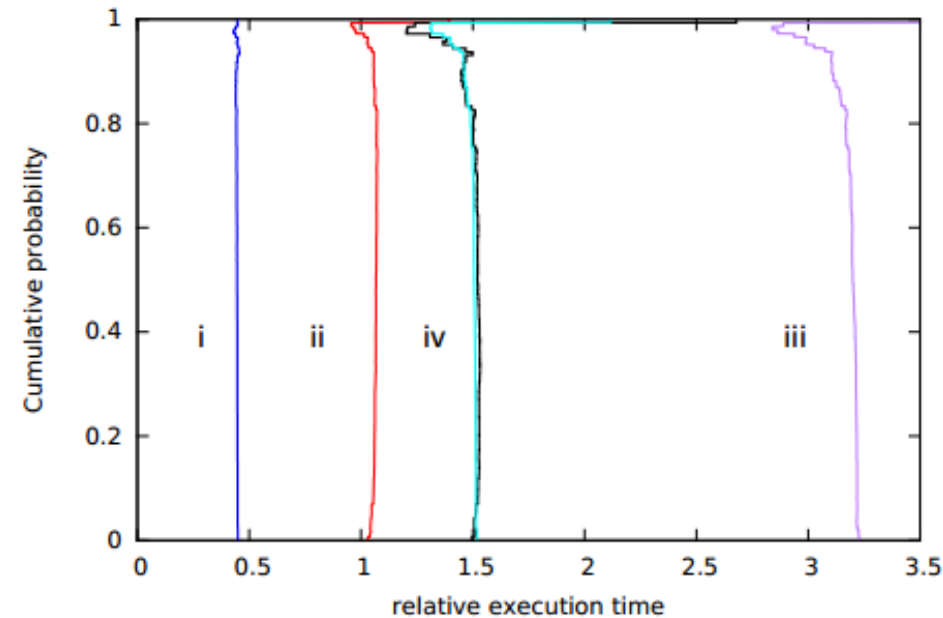
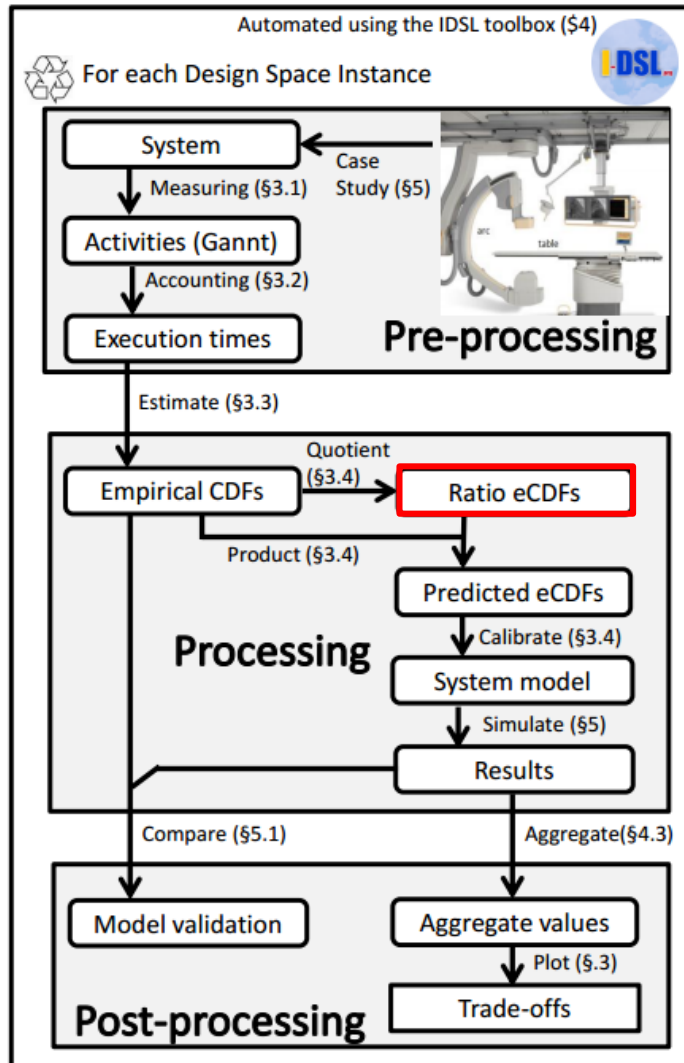
iDSL solution chain of the approach



Ratio eCDFs comprise ratios of empirical CDFs

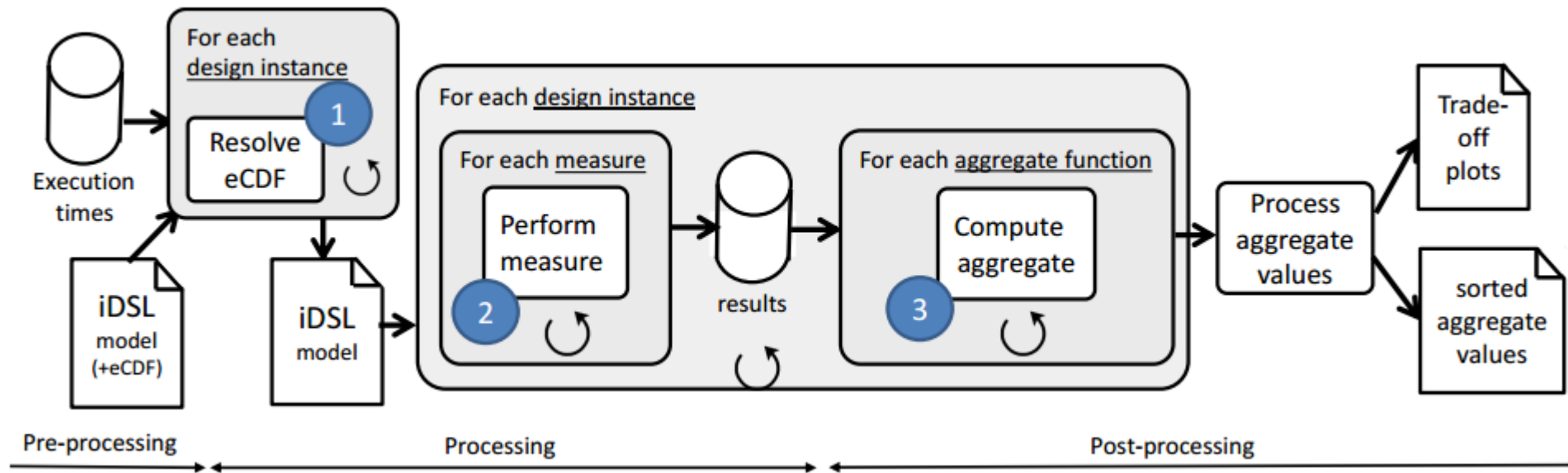
$$R(p) = \frac{Q_{f_1, mono, 512^2}(p)}{Q_{\bar{b}}(p)} \cdot \frac{Q_{\Omega, bi, 512^2}(p)}{Q_{\bar{b}}(p)} \cdot \frac{Q_{\Omega, mono, 1024^2}(p)}{Q_{\bar{b}}(p)}$$

iDSL solution chain of the approach



- i) Function f_1 takes 45% of the total execution time
- ii) Biplane systems, with better hardware, perform as monoplane ones
- iii) A 1024^2 image resolution leads to 3.2 times as much execution time
- iv) — The product of i, ii and iii (prediction based)
- The ratio for design f_1 , biplane, 1024^2 (measurement based)

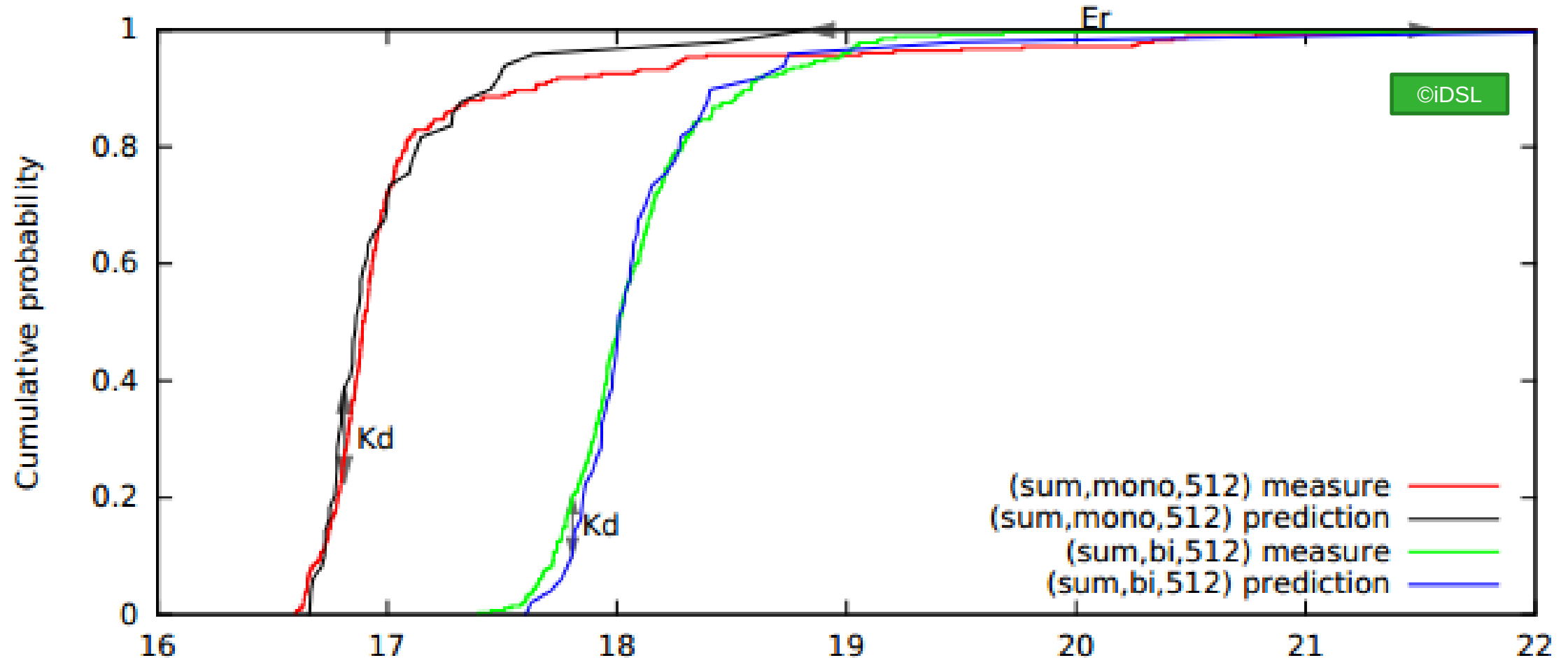
The solution chain of iDSL



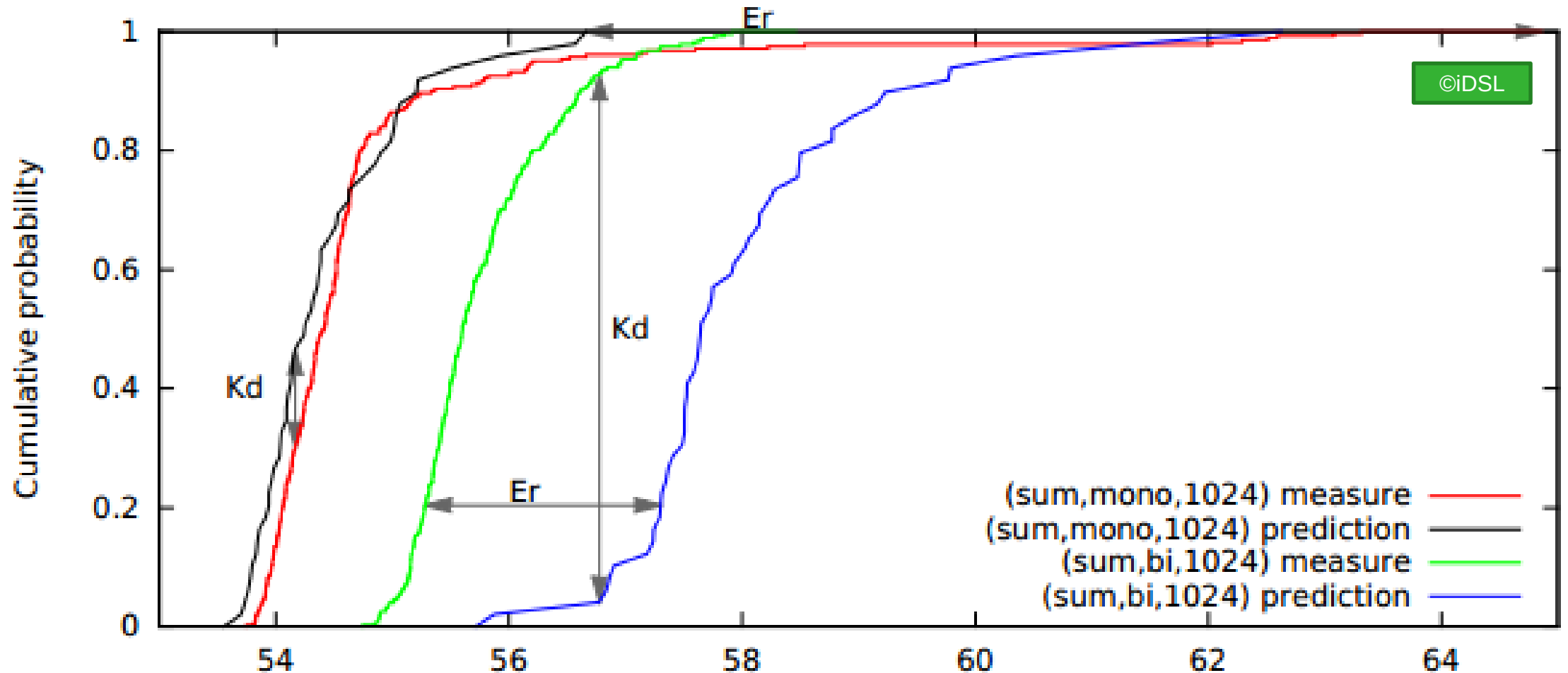
iDSL has been extended with

- eCDF support and operations, i.e., product, divide (1)
- Aggregated measures, e.g., average latency (3)

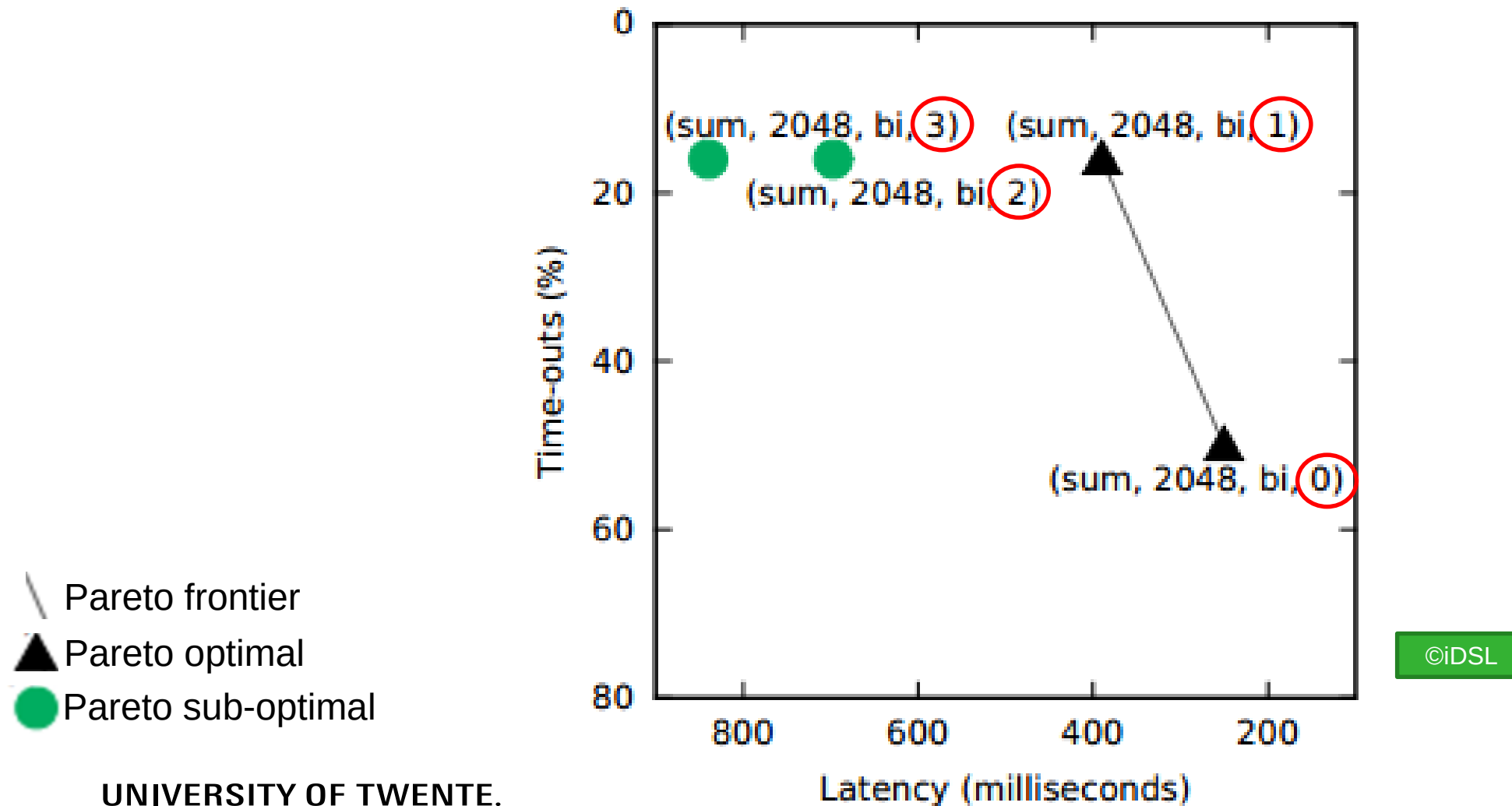
Results: Latencies for 512^2 pixels



Results: Latencies for 1024^2 pixels



Results: Buffer size trade-off



Conclusions

- We have extended iDSL to predict the performance
 - of many designs
 - using few measurements
 - returning understandable results, in a fully automatic manner

Conclusions

- We have extended iDSL to predict the performance
 - of many designs
 - using few measurements
 - returning understandable results, in a fully automatic manner
- We have validated our approach
 - For monoplane systems: the predictions reflect reality
 - For biplane systems with higher resolutions: predictions are pessimistic

Conclusions

- We have extended iDSL to predict the performance
 - of many designs
 - using few measurements
 - returning understandable results, in a fully automatic manner
- We have validated our approach
 - For monoplane systems: the predictions reflect reality
 - For biplane systems with higher resolutions: predictions are pessimistic
- We evaluated the performance of iXR machines
 - Biplane iXR systems perform as good on shared hardware as on dedicated hardware
 - The buffer size should be 0 or 1 images

Thank you for your attention



*

* Google Images: Spanish lunch

Outline

- Previous work: The iDSL language and tool chain
- Paper 1: Automated Performance Prediction of Medical Systems
- LUNCH
- Paper 2: Computing Response Time Distributions via Model Checking*
- Future work: ``Computing Response Time Distributions 2.0''

Context

- iXR machine
 - Again, real measurements are available
- Insights in performance needed
 - Key questions:
 - Q1: What is the latency for which a given percentage of service requests completes?
 - Q2: Which percentage of the service requests has a latency below a given value?
 - Simulations show average behavior, while WCET methods are pessimistic

Context

- iXR machine
 - Again, real measurements are available
- Insights in performance needed
 - Key questions:
 - Q1: What is the latency for which a given percentage of service requests completes?
 - Q2: Which percentage of the service requests has a latency below a given value?
 - Simulations show average behavior, while WCET methods are pessimistic
- Approach
 - Use probabilistic model checking, iteratively, to generate latency CDFs
 - Two model simplification strategies

iDSL model: process, resource, system

Section Process

```
ProcessModel Image_Processing mutex { seq {  
    seq Noise_reduction {  
        atom Pre_processing load ...    // depends on resolution and mode  
        atom Basic load ...  
    }  
    ...  
}}
```

Section Resource

```
ResourceModel Biplane_PC decomp { atom CPU rate 1 }    connections {(CPU,CPU)}
```

Section System

```
Service Image_Processing_Service  
    Process Image_Processing  
    Resource Biplane_PC  
    Mapping assign { ( , * , CPU) } scheduling policy { (CPU, FIFO) }
```

Section Measure

```
Measure CDF of ServiceResponseTimes via PTA model checking  
Measure ServiceResponseTimes using 3 runs of 100 ServiceRequests
```

iDSL model: process, resource, system

Section Process

```
ProcessModel Image_Processing mutex { seq {  
    seq Noise_reduction {  
        atom Pre_processing load ...    // depends on resolution and mode  
        atom Basic load ...  
    }  
    ...  
}}
```

Section Resource

```
ResourceModel Biplane_PC decomp { atom CPU rate 1 }    connections {(CPU,CPU)}
```

Section System

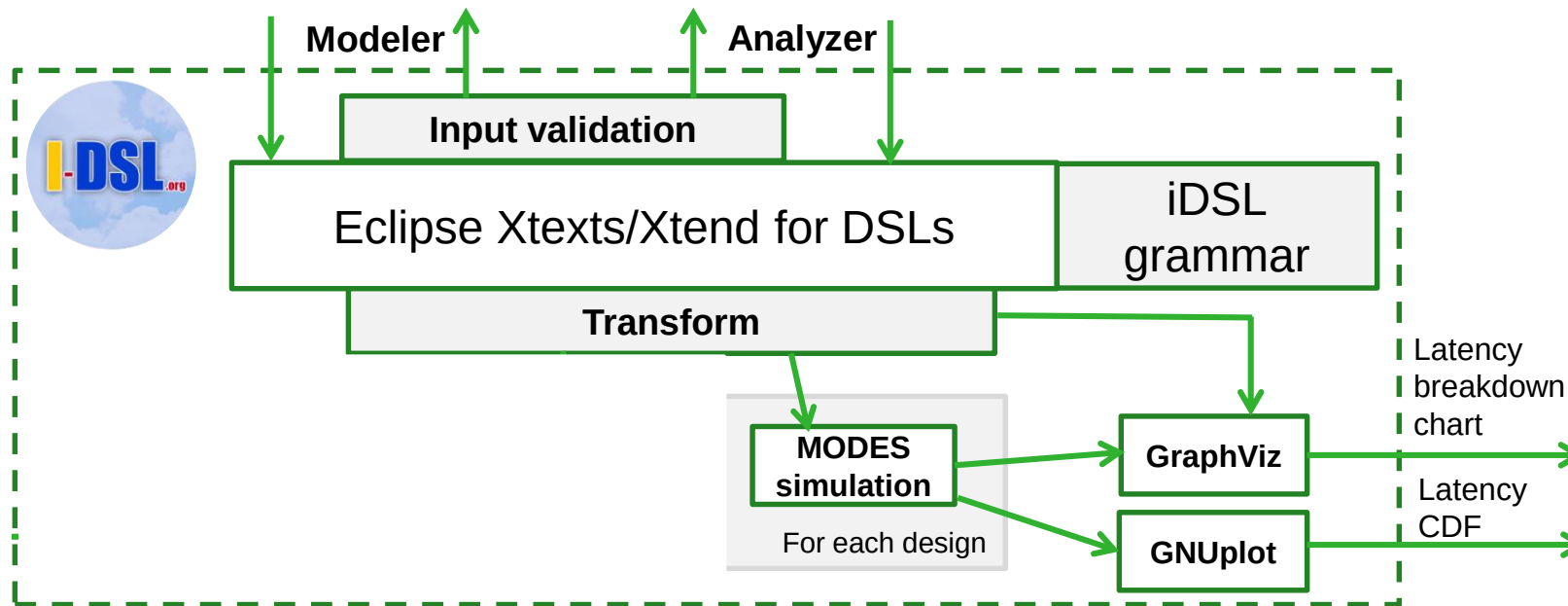
```
Service Image_Processing_Service  
    Process Image_Processing  
    Resource Biplane_PC  
    Mapping assign { ( , * , CPU) } scheduling policy { (CPU, FIFO) }
```

Section Measure

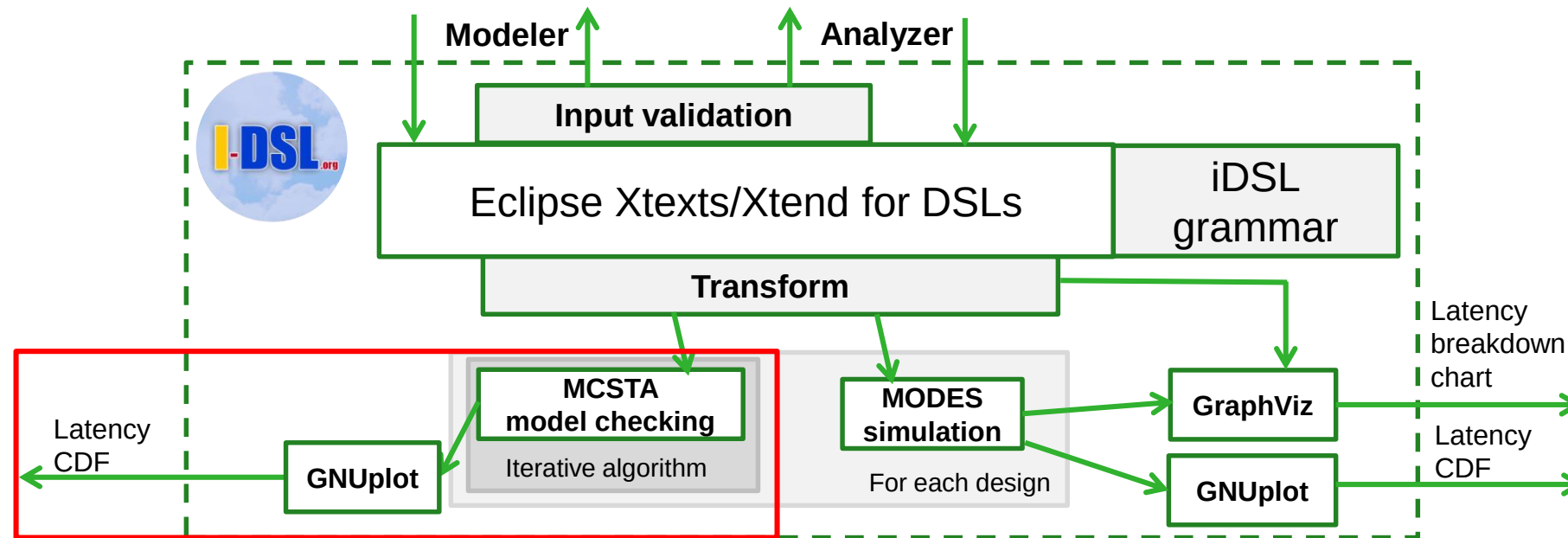
```
Measure CDF of ServiceResponseTimes via PTA model checking  
Measure ServiceResponseTimes using 3 runs of 100 ServiceRequests
```

One small step for the system designer, ...

iDSL solution chain



iDSL solution chain



Probabilistic model checking

- We define function $\mathcal{M}$ that returns the probability that a service finishes within a given time:

$$p = \mathcal{M}(i, s, f, t),$$

where i is an iDSL model, s a service, f a $p_{\min}/p_{\max}$ flag, and t the given time

Probabilistic model checking

- We define function $\mathcal{M}$ that returns the probability that a service finishes within a given time:

$$p = \mathcal{M}(i, s, f, t),$$

where i is an iDSL model, s a service, f a $p_{\min}/p_{\max}$ flag, and t the given time

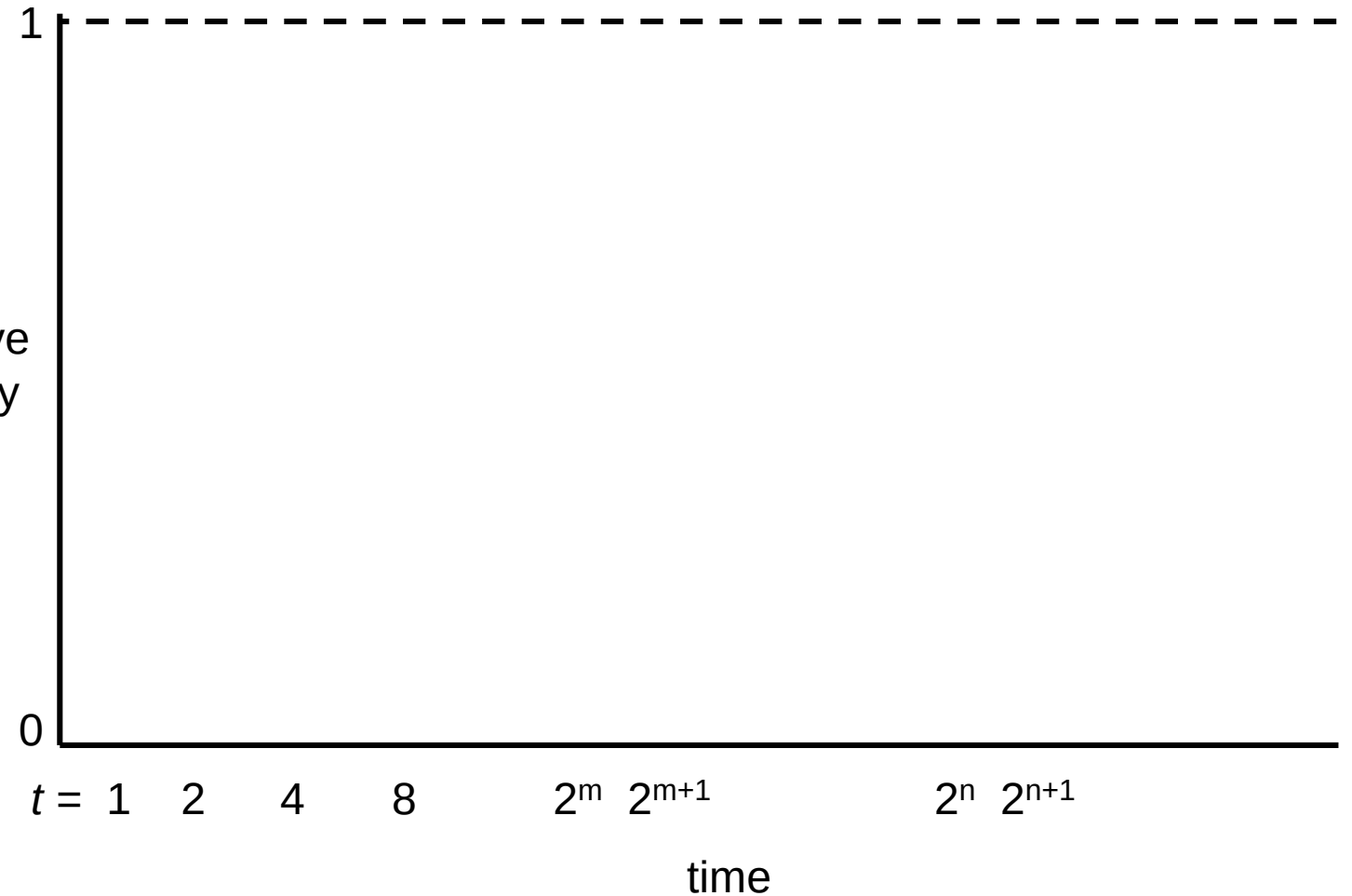
- We implement function $\mathcal{M}$ as follows:
 - 1) Generate the MODEST model for iDSL model i , service s , and flag f .
 - 2) Execute the model in MCSTA with parameter t
 - 3) Return the MCSTA result as probability p

Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

We now apply
function $\mathcal{M}$
for different t values

cumulative
probability

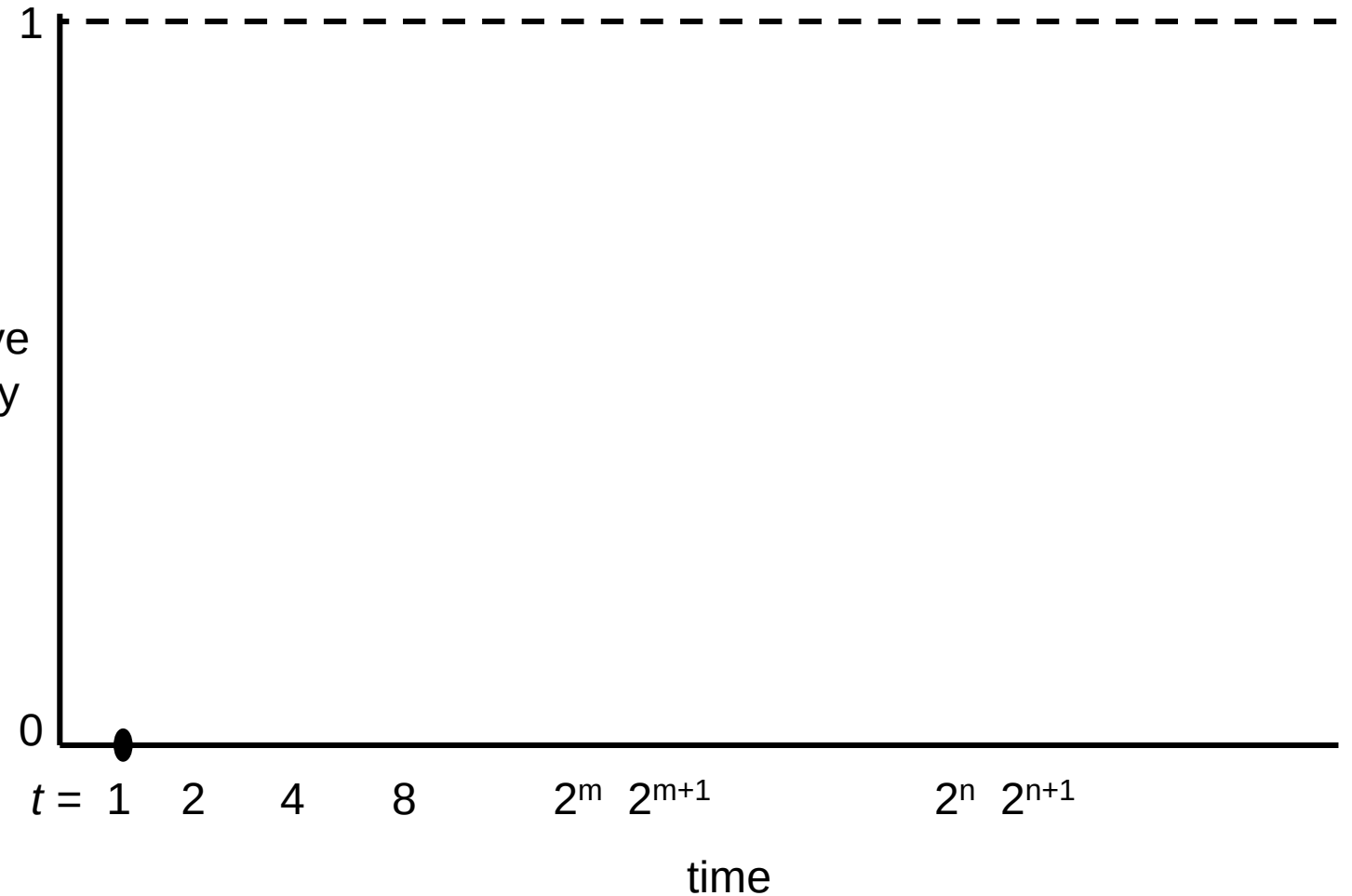


Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

We now apply
function $\mathcal{M}$
for different t values

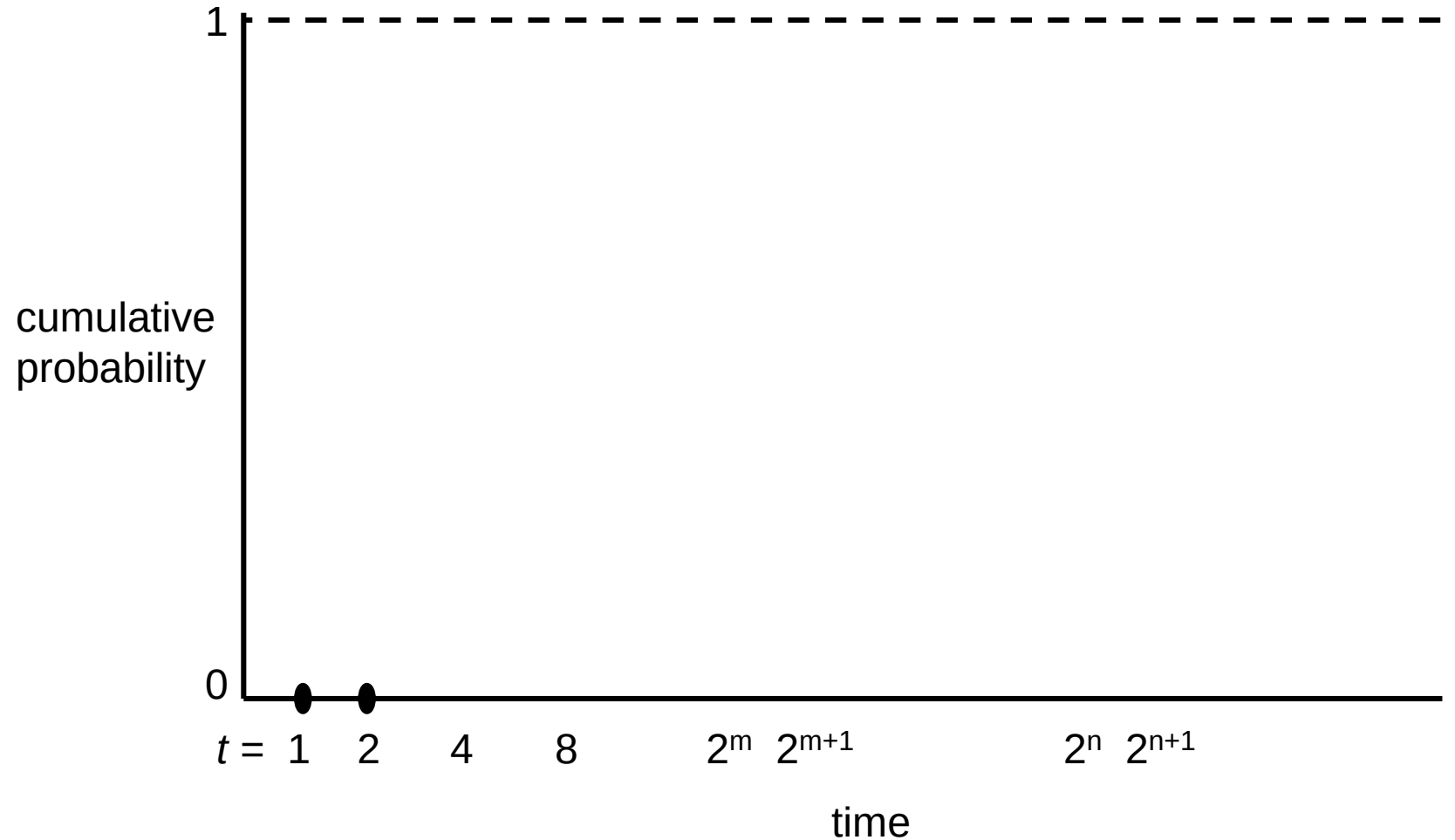
cumulative
probability



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

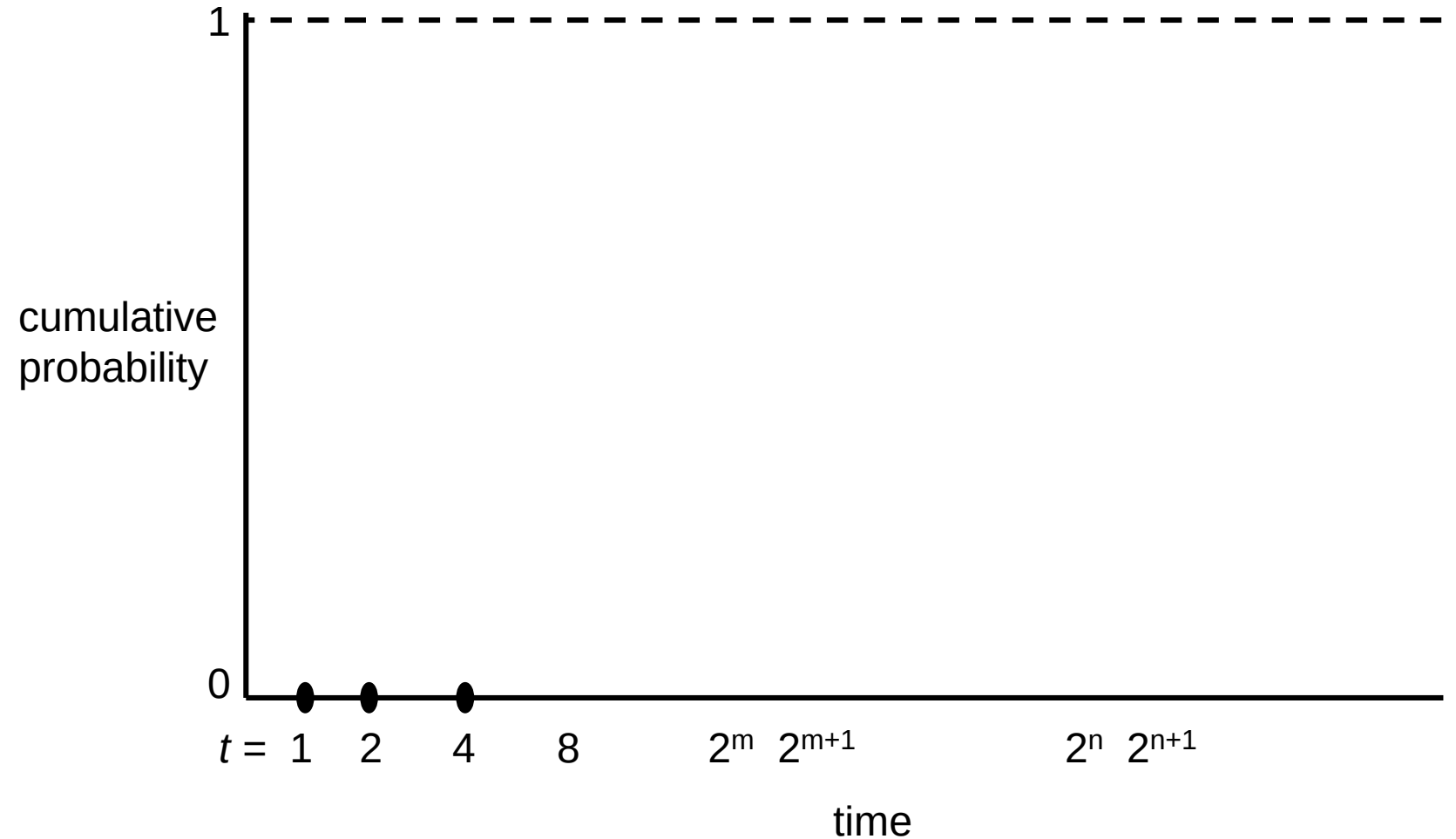
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

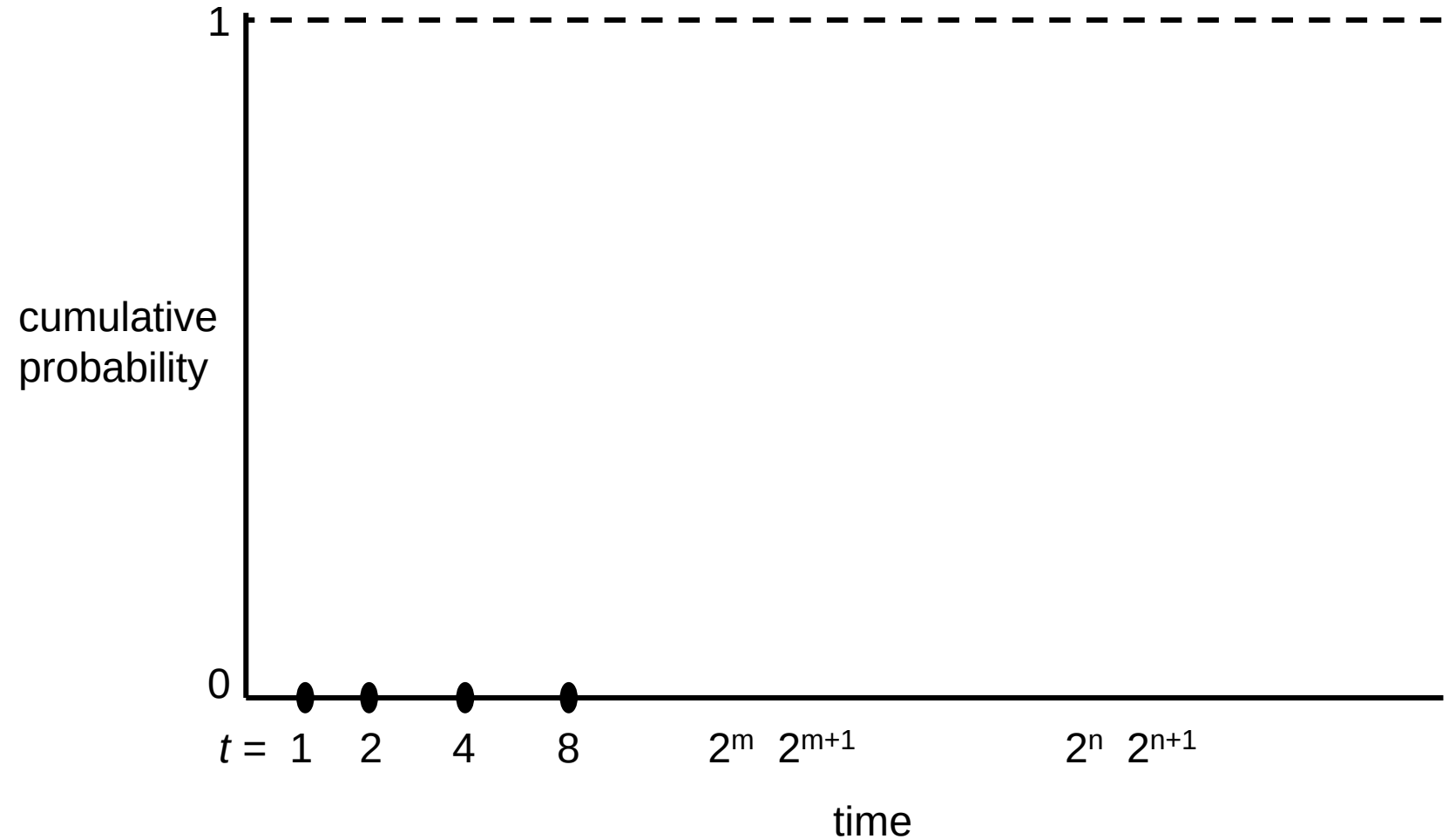
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

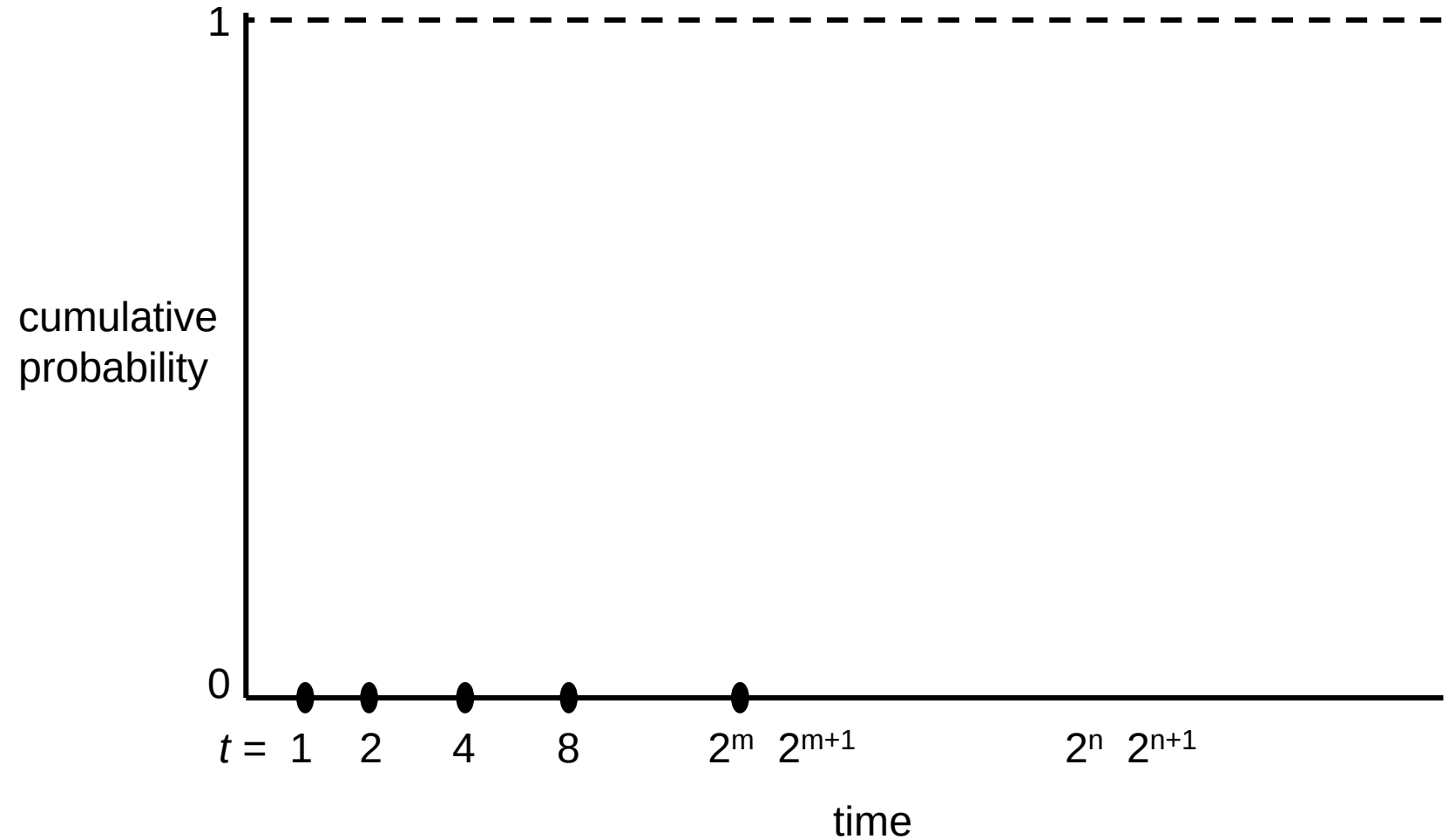
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

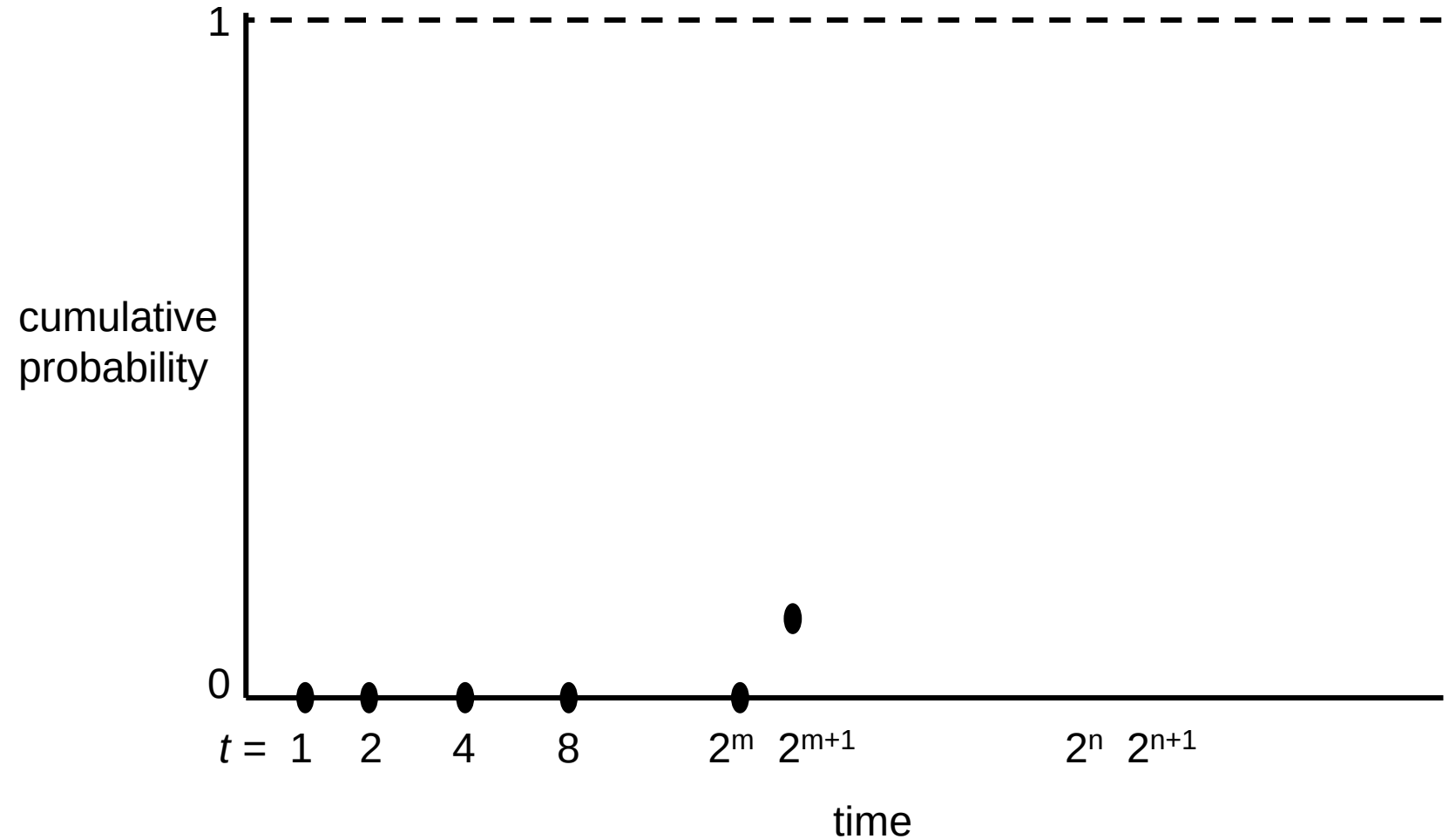
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

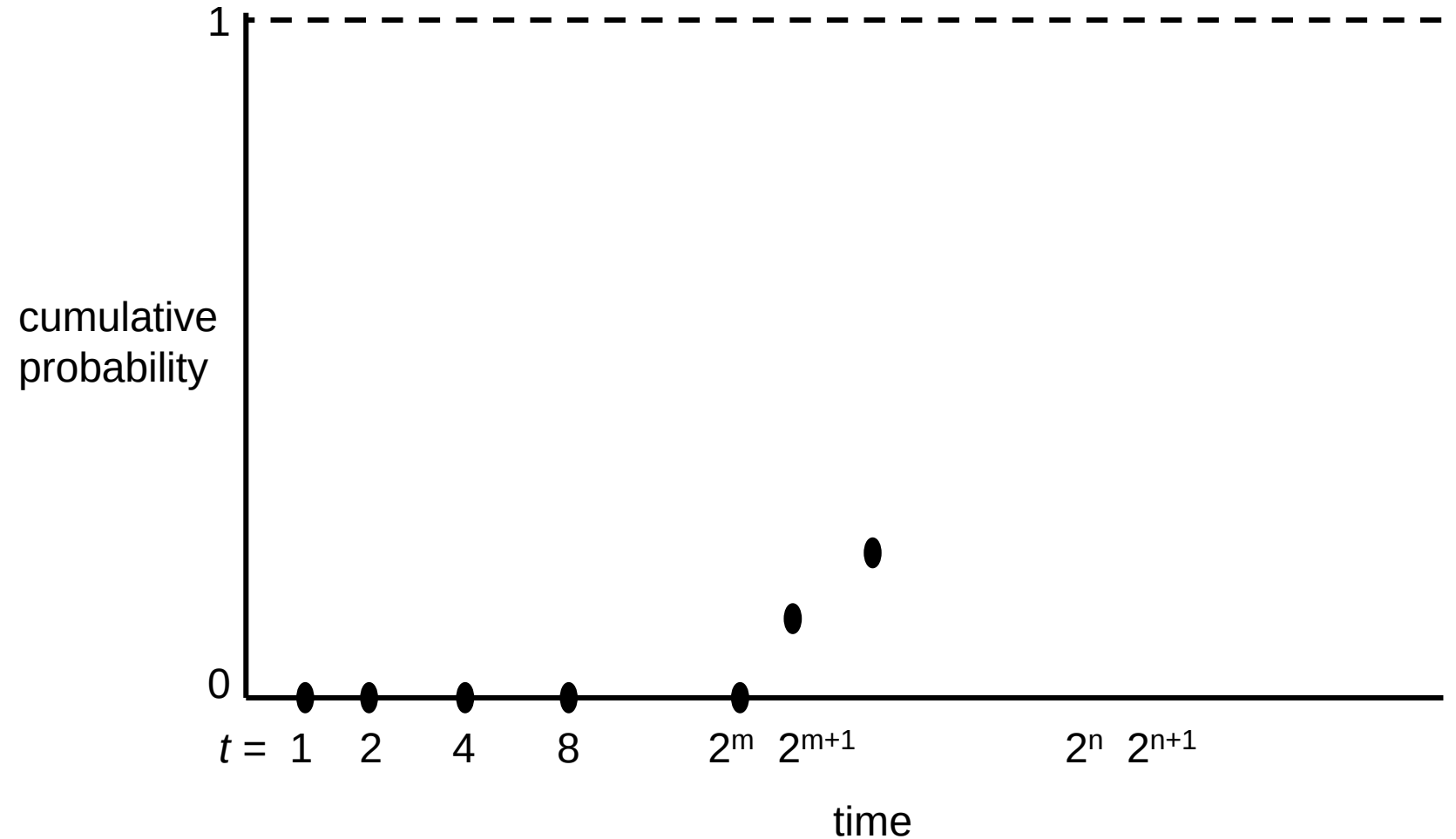
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

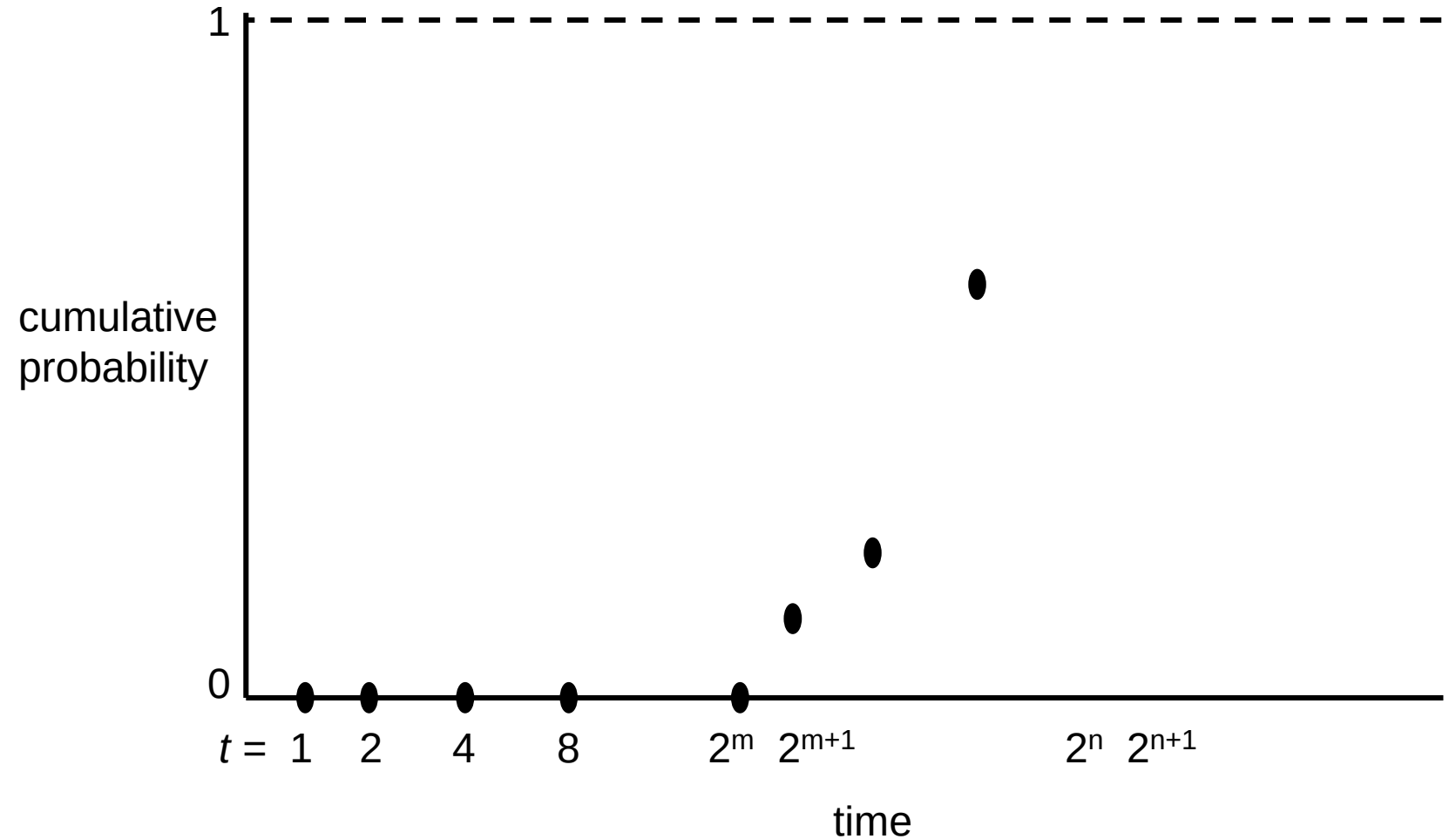
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

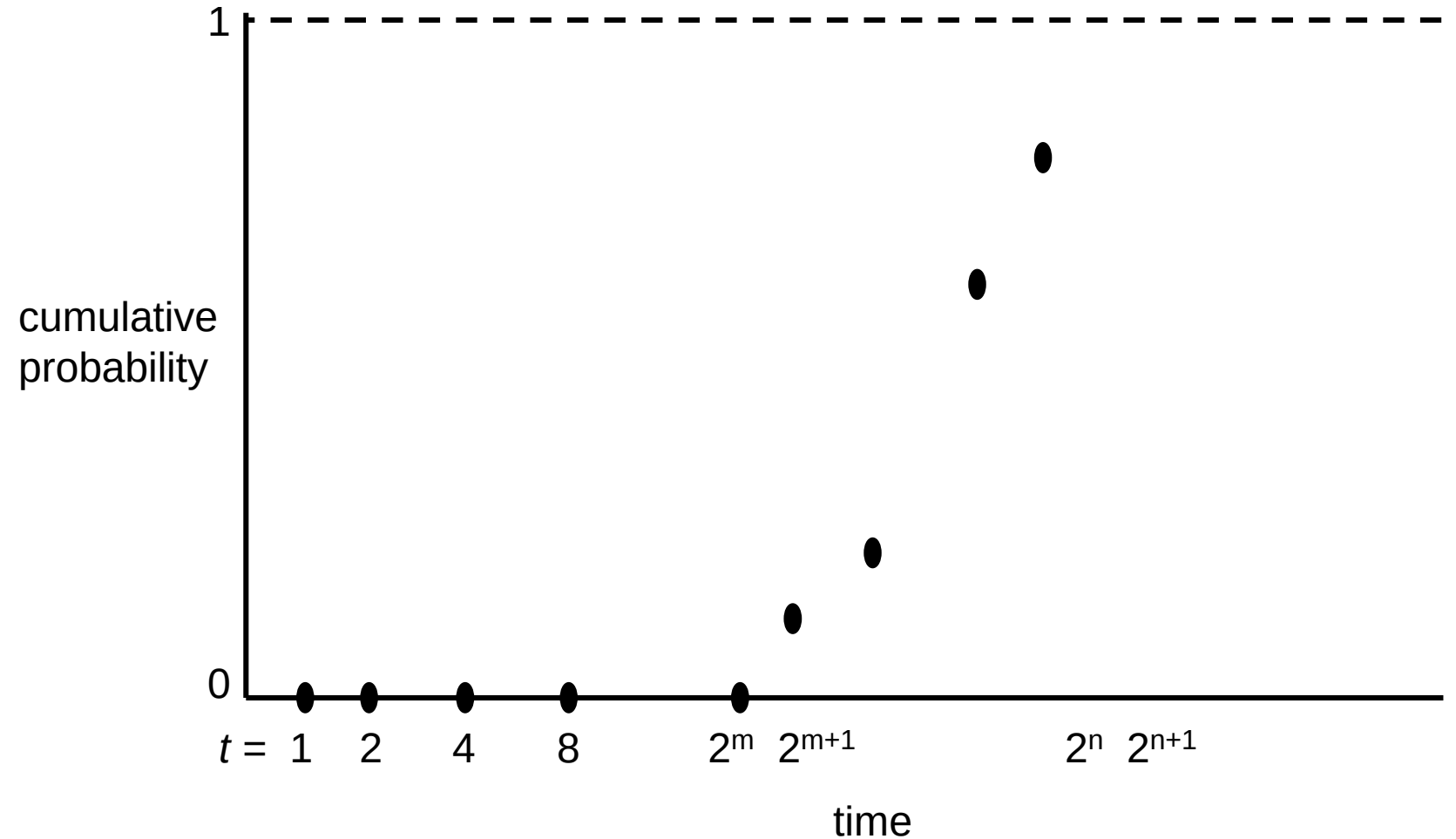
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

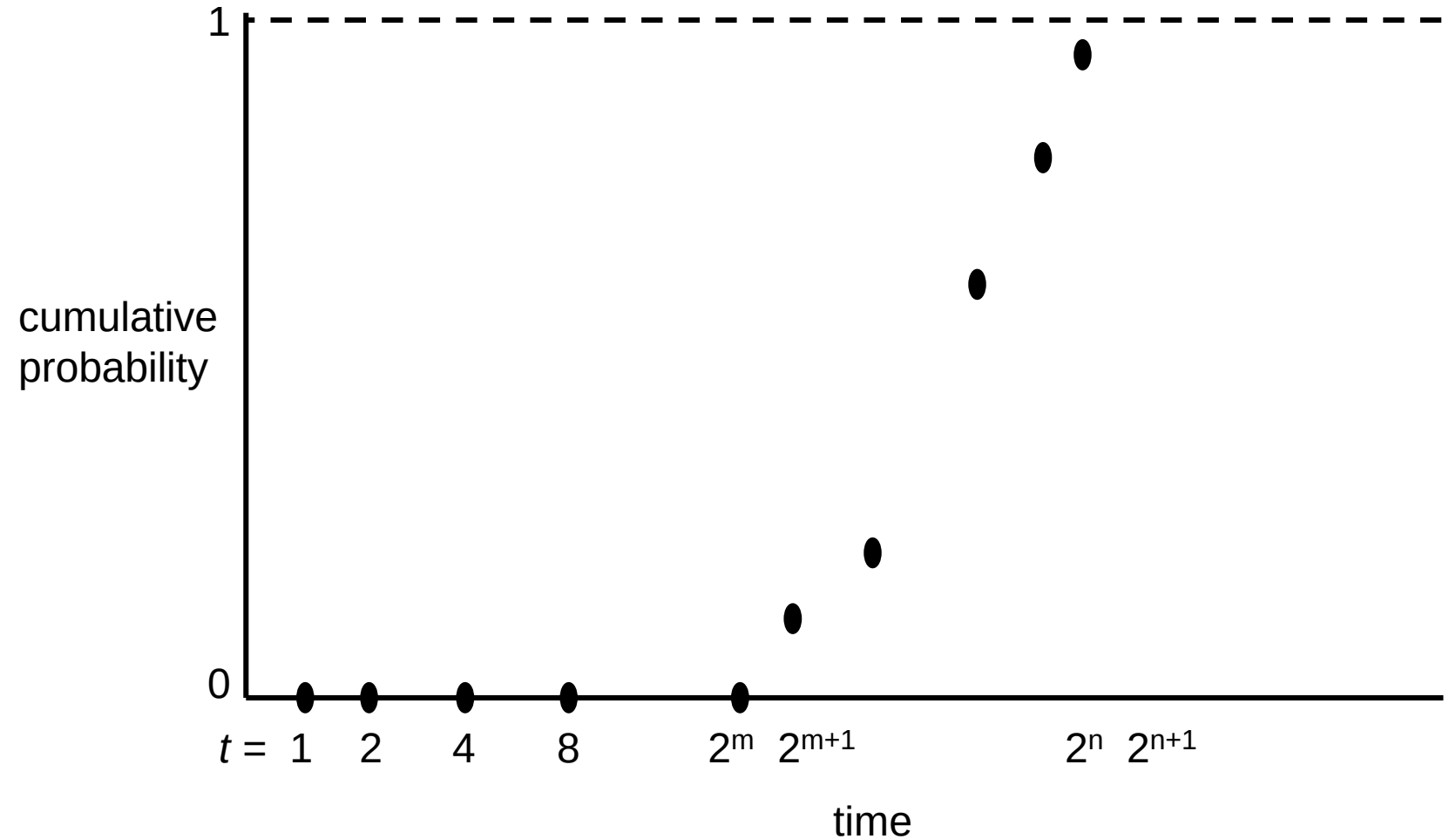
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

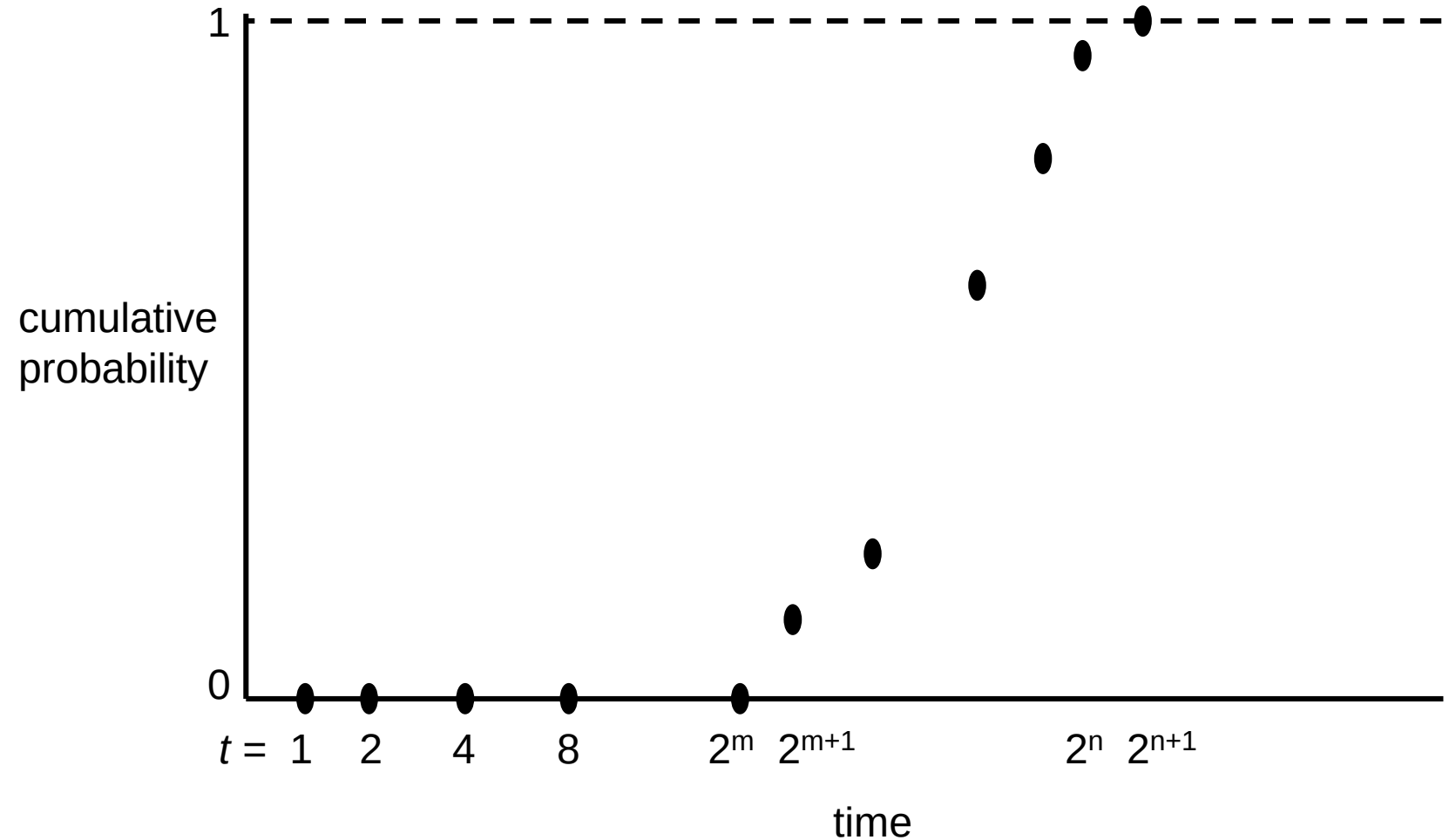
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

We now apply
function $\mathcal{M}$
for different t values

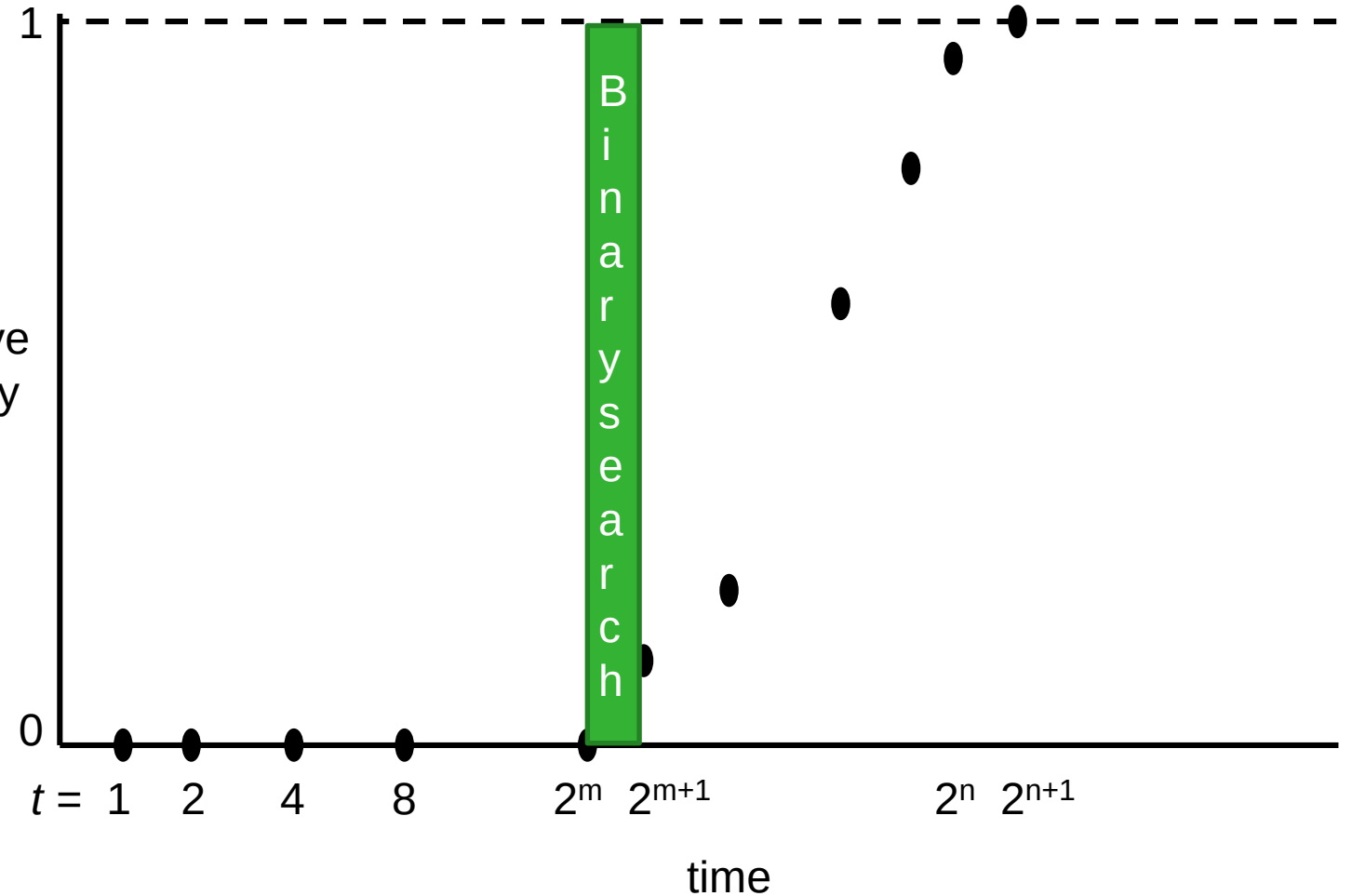


Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

We now apply
function $\mathcal{M}$
for different t values

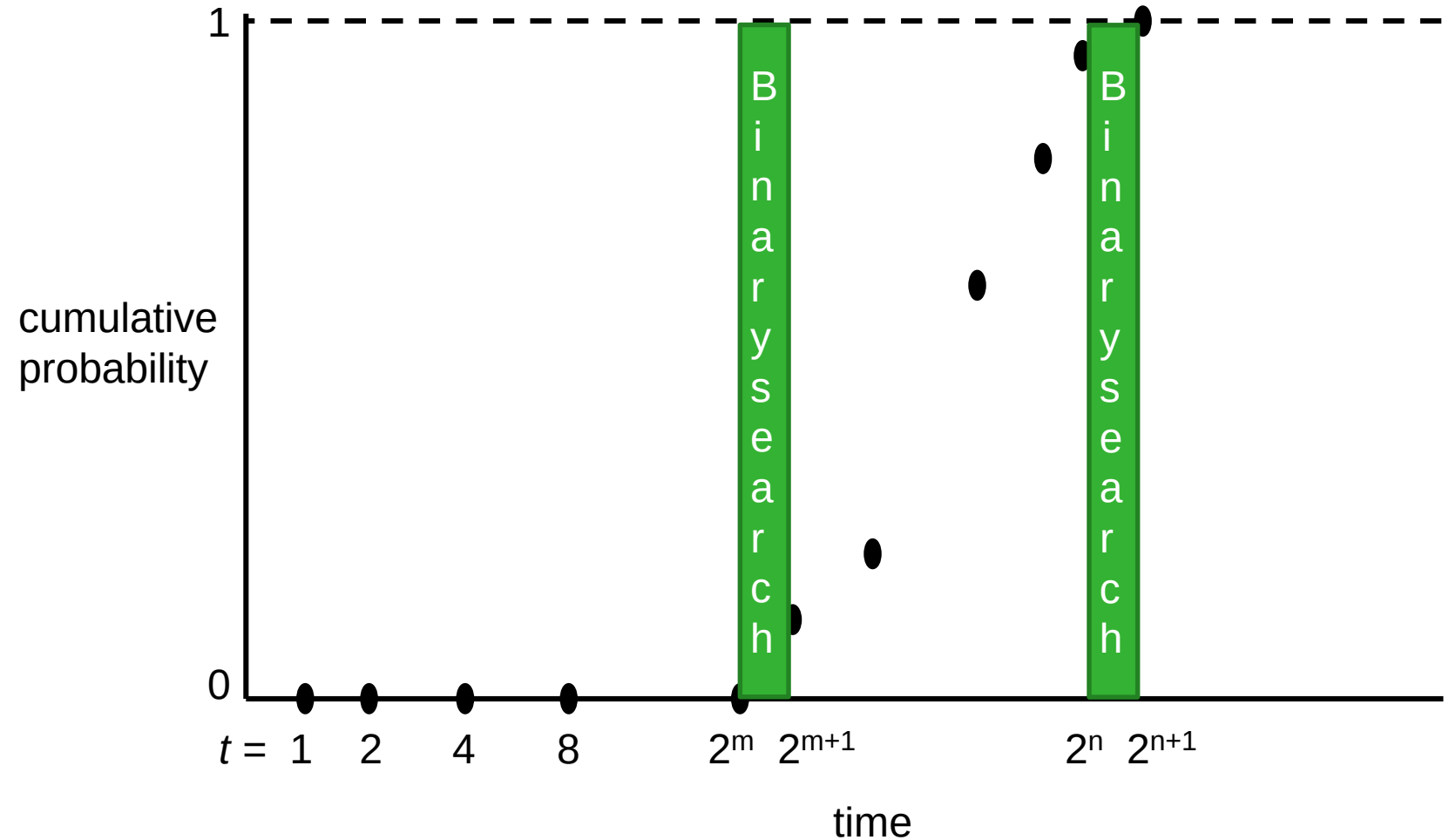
cumulative
probability



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

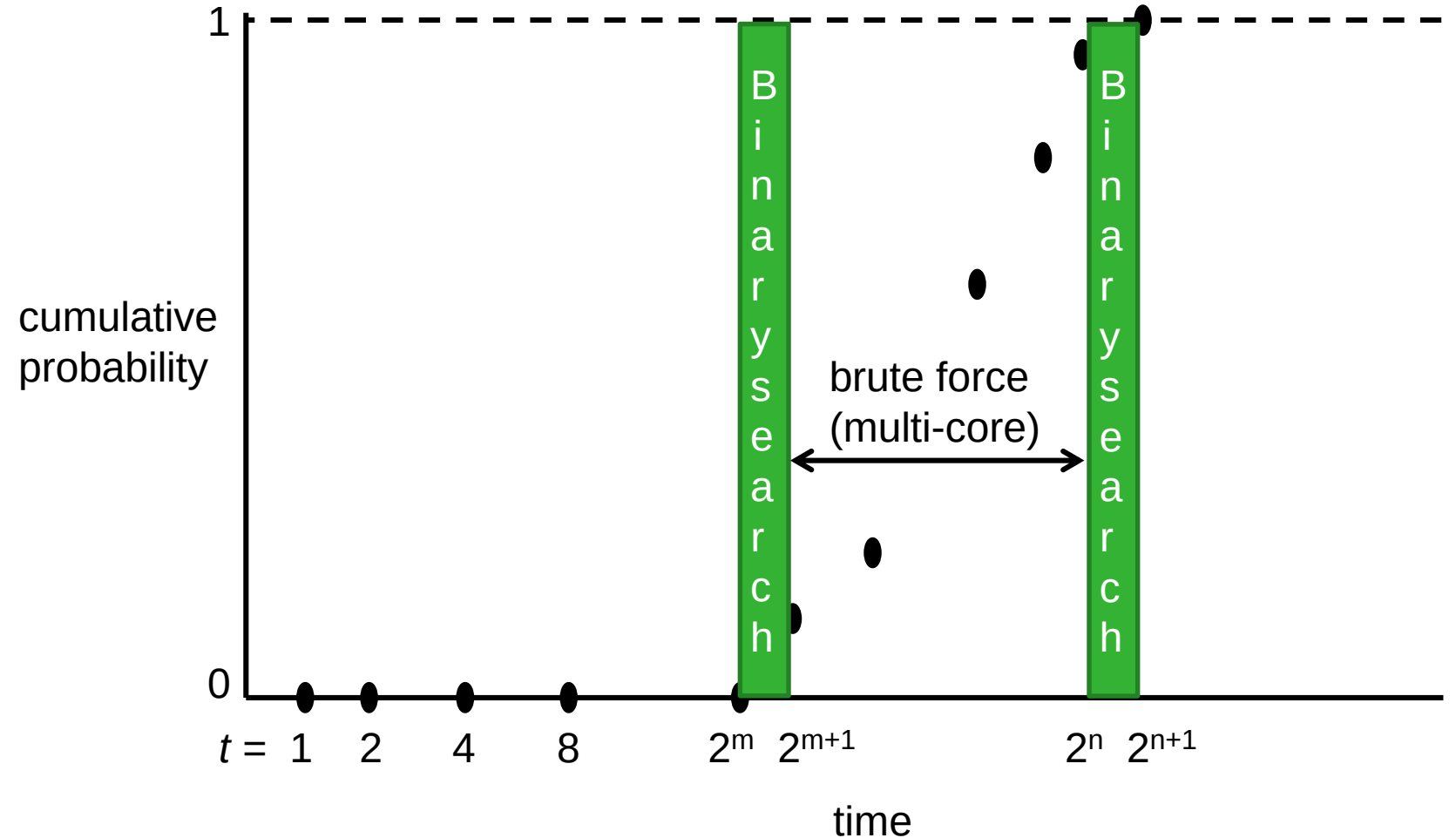
We now apply
function $\mathcal{M}$
for different t values



Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

We now apply
function $\mathcal{M}$
for different t values

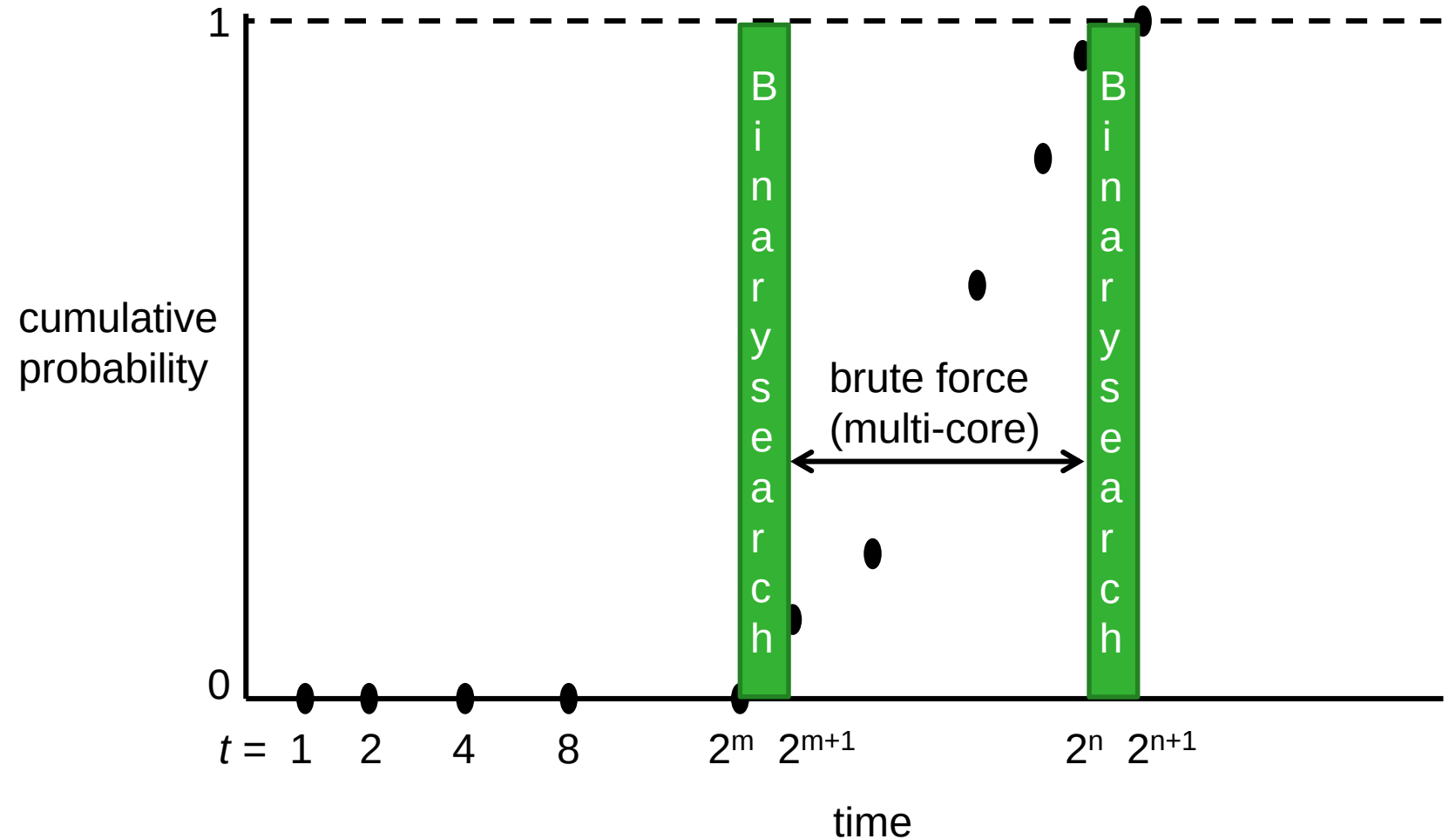


Probabilistic model checking, iteratively

$$p = \mathcal{M}(i, s, f, t),$$

We now apply
function $\mathcal{M}$
for different t values

And repeat this for
each model i ,
service s , and flag f .



Challenge #1: Infinite service requests

$P_n(t)$ = probability that service request n finishes within time t .

Challenge #1: Infinite service requests

$P_n(t)$ = probability that service request n finishes within time t .

- Ideally, we would like to know:

$$P_{\Omega}(t) = \lim_{k \rightarrow \infty} \frac{1}{k} \sum_{n=1}^k P_n(t),$$

Weight
Requests

Challenge #1: Infinite service requests

$P_n(t)$ = probability that service request n finishes within time t .

- Ideally, we would like to know:

$$P_{\Omega}(t) = \lim_{k \rightarrow \infty} \frac{1}{k} \sum_{n=1}^k P_n(t),$$

- Intuitively, we might like to compute:

$$P_{\Omega}(t) = \frac{1}{N} \sum_{n=1}^N P_n(t),$$

Weight
Requests

Challenge #1: Infinite service requests

$P_n(t)$ = probability that service request n finishes within time t .

- Ideally, we would like to know:

$$P_{\Omega}(t) = \lim_{k \rightarrow \infty} \frac{1}{k} \sum_{n=1}^k P_n(t),$$

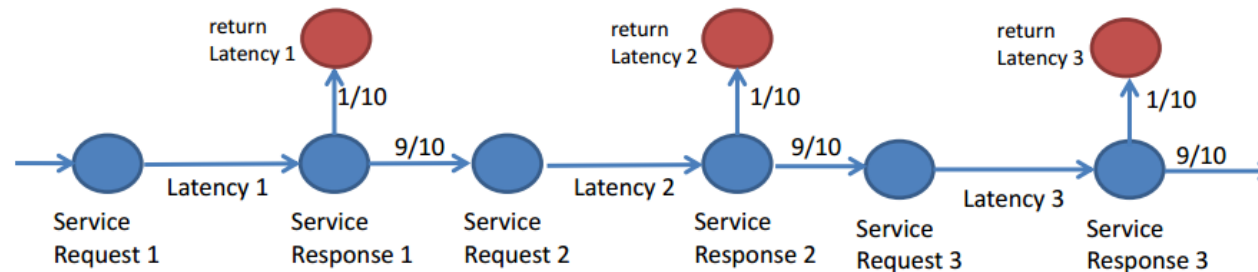
Weight
Requests

- Intuitively, we might like to compute:

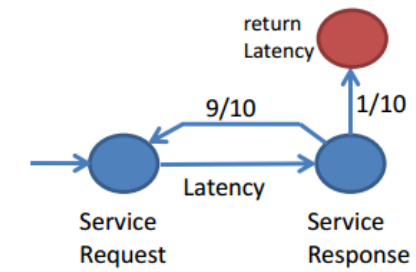
$$P_{\Omega}(t) = \frac{1}{N} \sum_{n=1}^N P_n(t),$$

- However, we choose to compute:

$$P_{\Omega}(t) = \sum_{n=1}^{\infty} (1 - \rho)^{n-1} \rho P_n(t),$$



(a) with a service request counter



(b) without a counter

Challenge #2: The state space is large

- Experiment 0: status quo
 - Leads to perfect results, in theory
 - But results into a state space explosion, in practice

Challenge #2: The state space is large

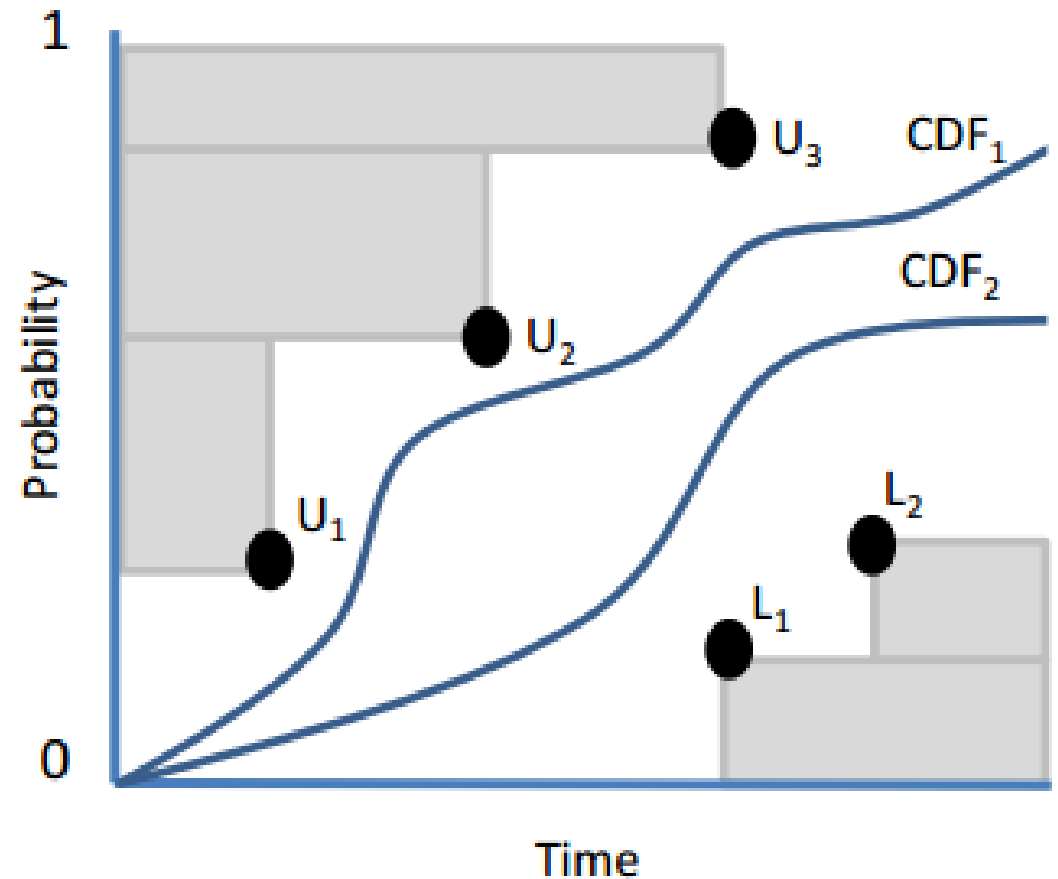
- Experiment 0: status quo
 - Leads to perfect results, in theory
 - But results into a state space explosion, in practice
- Experiment 1: increase the model time unit
 - Reduces the state space
 - Loss of precision

Challenge #2: The state space is large

- Experiment 0: status quo
 - Leads to perfect results, in theory
 - But results into a state space explosion, in practice
- Experiment 1: increase the model time unit
 - Reduces the state space
 - Loss of precision
- Experiment 2: replace probabilistic choices by non-determinism
 - Reduces the state space
 - Loss of information

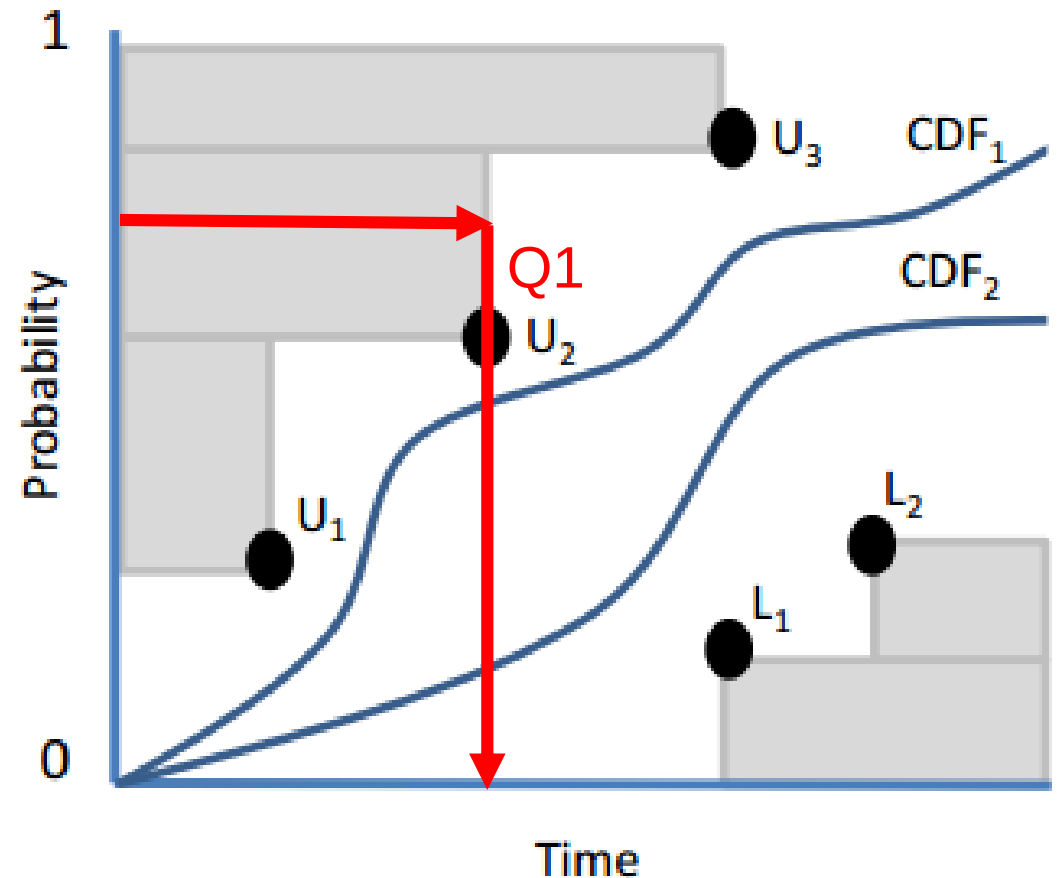
Post-processing

- Model checking results represented as an infinite set of eCDFs
 - U_1, U_2, U_3, L_1 and L_2 are function $\mathcal{M}$ outcomes



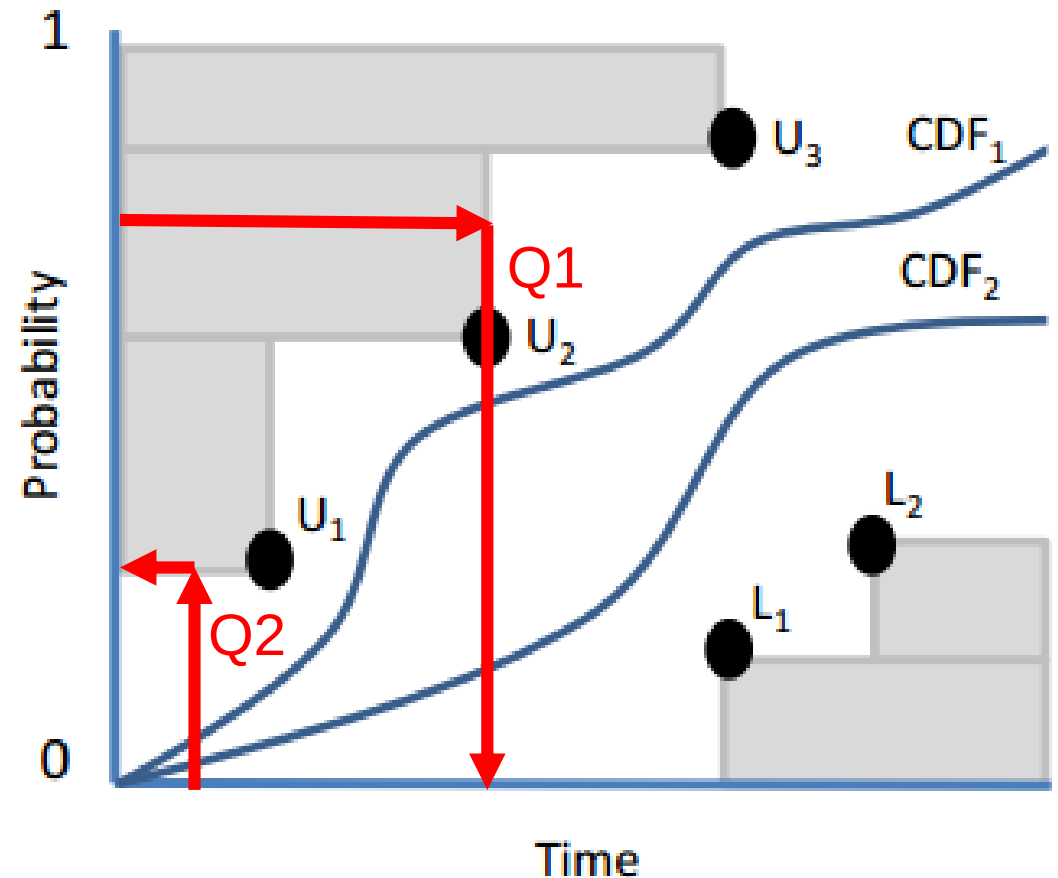
Post-processing

- Model checking results represented as an infinite set of eCDFs
 - U_1, U_2, U_3, L_1 and L_2 are function $\mathcal{M}$ outcomes
- Performance questions answered using this set
 - Q1: What is the latency for which a given percentage of service requests completes?

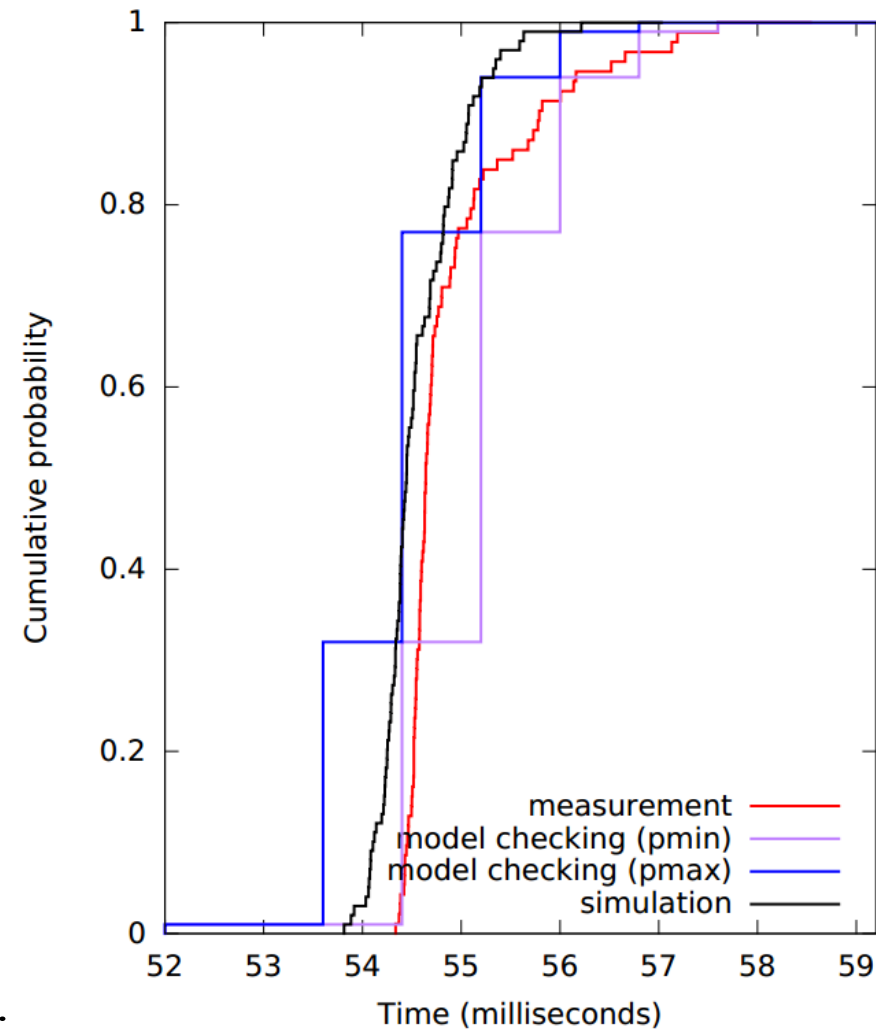


Post-processing

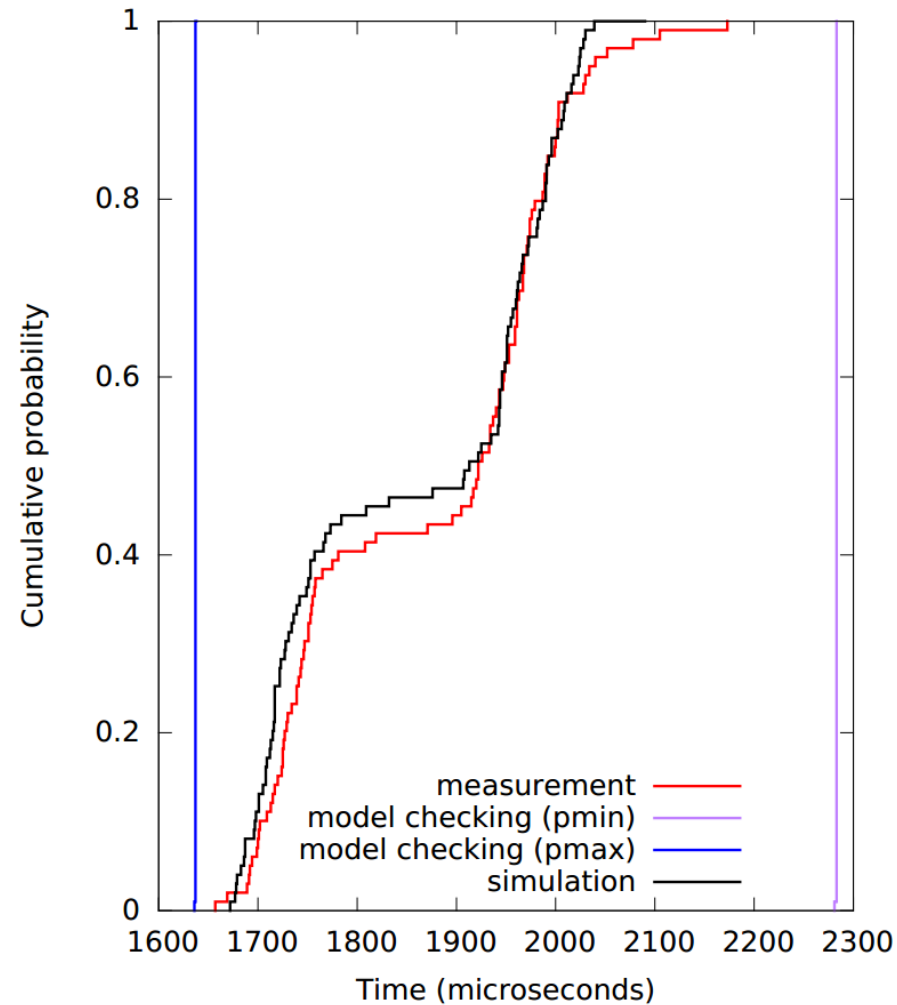
- Model checking results represented as an infinite set of eCDFs
 - U_1, U_2, U_3, L_1 and L_2 are function $\mathcal{M}$ outcomes
- Performance questions answered using this set
 - Q1: What is the latency for which a given percentage of service requests completes?
 - Q2: Which percentage of the service requests has a latency below a given value?



Results: Experiment 1 (increase the model time unit)



Results: Experiment 2 (replace probabilistic choices by non-determinism)



Conclusions

- We have added another measure to iDSL
 - Based on probabilistic model checking
 - Comprising an iterative algorithm
 - Capable of dealing with probabilistic choices and nondeterminism

Conclusions

- We have added another measure to iDSL
 - Based on probabilistic model checking
 - Comprising an iterative algorithm
 - Capable of dealing with probabilistic choices and nondeterminism
- This measure returns exact latency eCDFs
 - To answer questions like: Which percentage of the service requests has a latency below a given value?

Conclusions

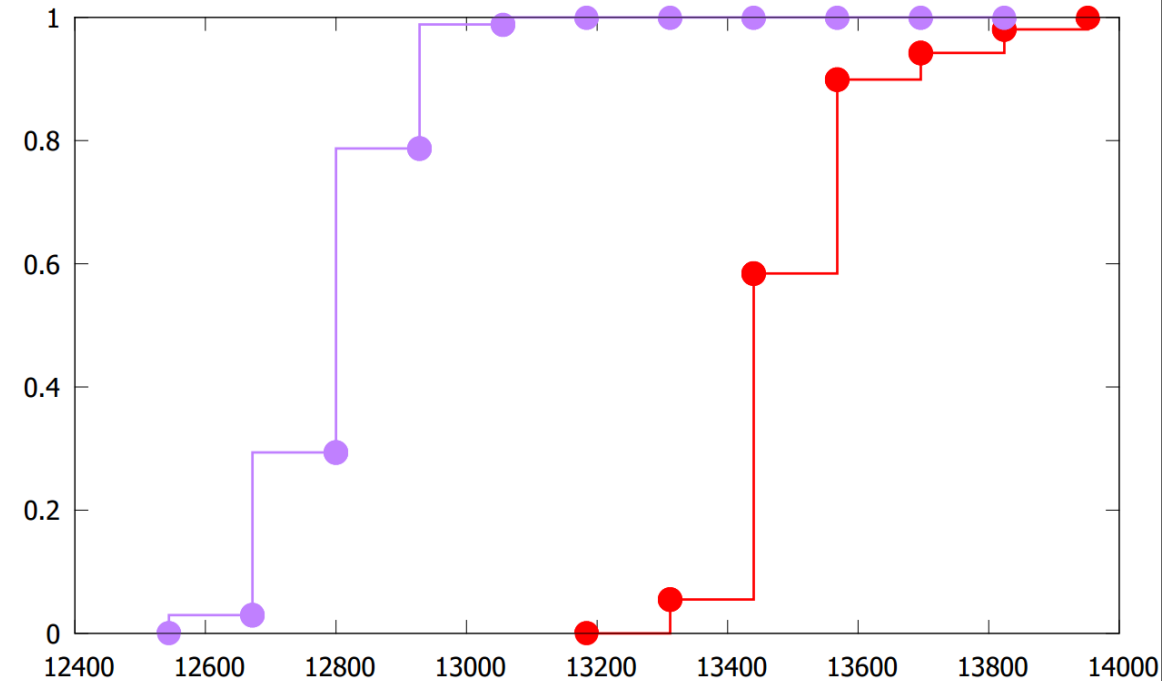
- We have added another measure to iDSL
 - Based on probabilistic model checking
 - Comprising an iterative algorithm
 - Capable of dealing with probabilistic choices and nondeterminism
- This measure returns exact latency eCDFs
 - To answer questions like: Which percentage of the service requests has a latency below a given value?
- The measure is prone to state space explosion
 - Two model simplification strategies
 - One introduces a loss of precision, the other a loss of information

Outline

- Previous work: The iDSL language and tool chain
- Paper 1: Automated Performance Prediction of Medical Systems
- LUNCH
- Paper 2: Computing Response Time Distributions via Model Checking
- Future work: ``Computing Response Time Distributions 2.0''

“Computing Response Time Distributions 2.0”

- Automatic model simplification
 - benchmark-based
- Anytime algorithm
- More efficient
 - Combining measures: basic estimations, simulations, model checking, and probabilistic model checking



Thank you for your attention



*

* Google Images: Madrid